

SPOT as a companion in AEC industry project sites

Project Report



Abhishek Dilip Patil,
Matriculation Nr. – 127126

Niraj Kishore Ladhe
Matriculation Nr. – 125815

Guides:

Prof. Dr.-Ing. Christian Koch
Ronny Helbing M.Sc.

Contents

1	Introduction	5
1.1	Background and Motivation.....	5
1.1.1	Digitalization.....	5
1.1.2	Current Inspection methods	5
1.1.3	Automation in AEC industry with robots	5
1.1.4	Motivation.....	6
1.2	Problem Statement	6
1.3	Objectives.....	7
1.3.1	SPOT program with both autonomous and manual functionality:	7
1.3.2	Main SPOT manager class:.....	7
1.3.3	Fiducial tag detection, follow and react to different tag IDs:	7
1.3.4	Video Stream and priority to manual input:.....	7
2	Literature Review	9
2.1	Automation in Inspection methods	9
2.2	Applications of ROBOTS in the construction sites	9
2.3	Versatility of Legged Robots.....	10
2.4	Quadruped robots	10
2.5	Case Studies of Spot in construction industries	10
2.5.1	Critical Infrastructure: Spot Inspects Hamburg Bridge [6].....	10
2.5.2	Turner Construction Uses Ground Robots to Cut Inspection Time by Over 95 Percent [7].....	11
2.5.3	RATP: Inspections after dark [8]	11
2.5.4	Virginia Tech [9]	11
2.5.5	Acciona [10].....	12
2.5.6	Promerleau [11].....	12
2.5.7	Brasfield & Gorrie [12].....	12
2.5.8	Foster + Partners [13]	13
2.5.9	Swinerton [14]	13
3	Methodology.....	14
3.1	Theory and Documentation.....	14
3.2	Communication and Networking	15
3.3	Hardware	16
3.3.1	SPOT.....	16
3.3.2	Core I/O	18
3.3.3	Ricoh Theta Z1 360° camera [19].....	18
3.3.4	X-Box controller	20
3.3.5	Client System.....	20
3.3.6	Fiducial tags.....	21
3.4	Software	21
3.4.1	Microsoft Visual Studio Code (VS Code)	21
3.4.2	GitHub.....	22
3.4.3	Spot SDK	22
3.4.4	Python Programming language.....	22
4	Implementation and Test cases	23
4.1	System Architecture	23
4.2	Python program [20]	24
4.2.1	Run_cmd.py	24
4.2.2	Spot_control_manager.py	24
4.2.3	Fiducial_follow.py	27

4.3	Working of the Program	29
4.4	Test cases.....	32
4.4.1	Detecting fiducials using at the client end using AprilTag Library	32
4.4.2	Possible to descend stairs in manual mode, but tricky in fiducial follow.....	32
4.4.3	Usage of Core I/O for more advance interaction with surroundings	32
4.4.4	Testing with other equipment such as Ricoh Theta Z1 360° camera and Controllers	33
4.5	Results and Discussion.....	34
5	Conclusion and Future Work	34
5.1	Conclusion.....	34
5.2	Future Work.....	37
	Appendix.....	39

List of figures

Figure 1: Block Diagram showing three different level of robot's services. Core, robot and autonomy. The Base, Payloads, Control and Data blocks are the necessary services that are a must to connect with SPOT.....	15
Figure 2: gRPC is built on top of other networking protocols, as shown in the diagram above.	16
Figure 3: Collective view of all the cameras of SPOT	17
Figure 4: SPOT Core I/O payload	18
Figure 5: Ricoh Theta Z1 360° camera, which has the possibility of being used as the payload.....	19
Figure 6: X-Box controller (with wired and wireless functionality)	20
Figure 7: Fiducial tag with dimensions.....	21
Figure 8:High level System Architecture of the spot program	23
Figure 9: Code Organization. Implemented by considering the Software engineering principles.....	23
Figure 10: run_cmd.py. To initiate/run the program	24
Figure 11: spot_control_manager.py.....	26
Figure 12: fiducial_follow.py.....	28
Figure 13: Autowalk/ Most common autonomous applications, image source: Sora.....	35
Figure 14: Combination of autonomous fiducial follow and manual control, image source: Sora	36
Figure 15: Communication using gRPC (block diagram)	39
Figure 16: Unary Streaming RPC, image source: ChatGPT.....	40
Figure 17: Server Streaming RPC, image source: ChatGPT	41
Figure 18: Client Streaming RPC, image source: ChatGPT.....	42
Figure 19: Bi - directional RPC, image source: ChatGPT	43
Figure 20: Working with RPC and SDK imports, image source: ChatGPT	44
Figure 21: Flow chart of the process at the client end	44
Figure 22: SPOT axes that are used for motion definition	45
Figure 23: Front view of SPOT	46
Figure 24: Side view of SPOT	47
Figure 25: Basic labelled diagram of SPOT	48

List of Tables

Table 1: Key bindings for manual input	25
Table 2: basic function flow of the program	31

1 Introduction

1.1 Background and Motivation

1.1.1 Digitalization

The AEC (Architecture, Engineering and Construction) industry is evolving in today's time with "Industry 4.0". Like other industries, there is a shift from document-centric delivery to data-centric, mode-driven workflows that interconnect design, construction and operations. BIM is at the core, which provides a common data environment enriched by open standards (e.g. IFC) and linked to schedules and costs (4D/5D simulations). Field technologies – laser scanning, drones, quadruped robots and IoT sensors – feed continuous reality capture back into the model, enabling automated progress verification, quality checks, and safety monitoring. Structured data, APIs and cloud collaboration, teams move from periodic reporting to near-real-time updates with dashboards and predictive control, where deviations are detected early, and targeted interventions and design updates are timely propagated downstream. A lot of benefits have been observed with this change. It includes fewer clashes and RFIs, reduced rework, faster inspections, and better traceability. Although, new challenges such as data governance, interoperability, cybersecurity and organizational change remain, digitalization has helped perform more complex projects with multiple stakeholder seamlessly and cost effectively. [1]

1.1.2 Current Inspection methods

The current construction monitoring and inspection is manual process and has led to inefficient contributions to cost overruns in 66% of construction project and schedules delays by 53% in projects, affecting almost \$1.4 trillion worths of structures being constructed each year in United States. [2]

The absence of real time information handicaps manager's ability to monitor schedule, cost and other performance indicators, which reduces their ability to detect or manage the variability and uncertainty inherent in project activities. A significant portion of the inspection time (30-50%) is spent on visual data collection of the as-built status and on-site analysis. [1] Also, the quality of inspection data is also determined by the inspector's experience and the methods used to collect measurements. [2]

1.1.3 Automation in AEC industry with robots

Digitalization is the first part of shift in the way industry works. It opens the door to high level automation of tasks. Many repeatable tasks are tedious for humans and affect the precision and accuracy. Automation of such tasks can better help the professionals to divert their focus on analysis rather than data collection, helping

organizations to take decisions faster. Robots with sensors and cameras mounted as payloads, can perform these tasks with higher precision and no fatigue. Also, robots can enter hazardous environments, and the operator can inspect the area without any risk. [3]

1.1.4 Motivation

The above benefits motivated us to look at solutions that involved a quadrupled robot. Quadrupled robots are more versatile in a construction site due to the design being close to the natural evolution of mammals. Although, there are many case studies and research on the use of robots at construction sites. In autonomous operations, most of those applications have a limitation of only having a prerecorded path that a robot can follow. While performing these operations, manual intervention in the movement is still felt necessary, due to the complex environment of the site. But the autonomous program should be completely exited to switch to manual mode which is not a user-friendly way and has high lead time. We wanted to come up with a solution that can bridge the gap where autonomous feature can keep on working but also accept manual intervention when felt necessary by the professionals. An indirect comparison to having a trained dog by the side to perform certain tasks on its own but also obey to some commands given by the trainer.

1.2 Problem Statement

The main challenge of this project is to make SPOT navigate by actively interacting with the surrounding – specifically, by continuously scanning the environment for fiducial tags – and to use those detections to drive motion decisions instead of following a pre – defined path. This requires reliable tag detection and robust decision logic so SPOT can approach, confirm, and act on tags despite occlusions, motion or variable lighting. In parallel, a layer of manual oversight must always be available, with access to features ranging from emergency stop to movement of SPOT. Thus, an operator can intervene safely and intentionally at any time. Implementation of these features requires clear understanding of the basic operation of SPOT, communication pathway between SPOT and the client system, including how commands, status and feedback are exchanged and synchronized. It also requires gaining knowledge on the various payloads available for SPOT for better operation and data handling. Equally important is proper handling of the errors and response signals to avoid system crashes. Also looking for proper open-source libraries compatible with the SPOT API to fill the gaps needed for functioning of the code to ensure stable integration and maintainable code

1.3 Objectives

The main objectives in the project are:

1.3.1 SPOT program with both autonomous and manual functionality:

Integrating the manual functionality with fiducial detection is a challenge as simultaneous access can lead to runaway between requests, RPC timeout errors or erratic behaviour from SPOT. Also, heavy logging overloads the network which can again lead to laggy response. A seamless integration of sensor data transfer and essential command signals for robust performance. Keeping a basic but user-friendly interface for clear understanding of the state of SPOT and better handling. And at last, using minimal computation power while autonomous functioning is important for lag-free operation.

1.3.2 Main SPOT manager class:

Creating a main class that has control over the essential features like the emergency electronic stop (ESTOP), motor power, establishing valid network connection between the SPOT and the client system, verifying that the connection is secure. Then, along with the essential services, also having decent manual control on the movement of the SPOT for manual navigation through tight spaces where the parameters set for collision detection might fail. Having a basic user interface to know the mode of the robot or see important information, e.g. battery percentage, status of motor power, network connection validity.

1.3.3 Fiducial tag detection, follow and react to different tag IDs:

A secondary class having features for fiducial detection using the body cameras present on SPOT and following the tag specified by the user. SPOT should be able to detect all the fiducial tags present in the detection range. Assigning a top priority to tag (master tag) where the SPOT will follow the tag if detected. Reacting to another tag ID (preferably a physical movement).

1.3.4 Video Stream and priority to manual input:

Stream the video from all the available cameras of SPOT for operation in places where it is difficult to have SPOT in the range of vision. Ensuring the stream is not lagging the actual movement by a large margin. This feature also helps to compare if a tag is in the visible range of SPOT.

Coding the program for manual input to have a higher priority over fiducial follow for safe recoveries of SPOT from difficult surroundings

2 Literature Review

2.1 Automation in Inspection methods

Previous studies investigated automating construction inspection. These studies aimed to increase the frequency and accuracy of inspection by capturing visual data using semi-autonomous or autonomous systems. These studies either focused primarily on the data capture alone and not on presenting the information in a way that is intuitive for inspection or used heavy post-processing that requires skilled personnel with specialized training. The key in progress monitoring is accurate, timely, and regular data collection and analysis during the construction process. In fact, automated progress monitoring of construction projects using emerging technologies can address existing limitations and inefficiencies of the manual processes and can enable systematic recording and analyzing construction progress. Identified four areas in automating the process of progress monitoring: (a) data collection from the actual construction work, (b) extracting required as-built data for analysis, (c) comparing the as-built data with as planned information and (d) visualizing and reporting the results.

Specifically in order to automate data collection in this process, studies have used various technologies, for example, a combination of Radio-Frequency Identification (RFID), laser scanning, photogrammetry with a digital camera, barcoding and a mobile device, a combination of camera-equipped UAV (Unmanned Aerial Vehicles), 4D Building Information Modeling (BIM), and 3D point cloud model and a data fusion framework that combines multiple data sources including 3D CAD model, ultra-wideband (UWB) receivers data, and laser scanned data. In this regard, collecting photo images of the actual construction status is very important for the reporting stage and for supporting communication of the identified issues with the stakeholders. 360° cameras are being utilized in these applications to increase visibility and decrease the capture time. Recent advances in automation have enabled robotic and autonomous inspection and monitoring. Mobile robots can be deployed to release the human from the laborious task of constant on-site image capture. [4]

2.2 Applications of ROBOTS in the construction sites

Autonomous navigation through the unstructured and dynamically changing construction environments requires mobile robots that can carry out high-performance locomotion tasks. Mobile robots use various types of locomotion to navigate an environment. Typical locomotion types used for mobile robots are : (a) wheeled robots using multiple wheels for navigation as the most stable robots with high payload capacity but they can operate on limited uneven surfaces and cannot move between levels through the stairs independently, (b) crawler-mounted robots using crawlers with continuous tread bases that can operate on unlevelled and unfinished ground, (c) climbing robots using suction mounts or magnetic adhesion to stick to walls that are typically used for façade inspection but their challenge is to stay affixed to the surface reliably while being able to avoid obstacles, (d) Unmanned Marine Vehicles (UMV) mechanically sealed to operate in water and typically used for underwater inspection

with special sensing devices, (e) Unmanned Aerial Vehicles (UAV) using rotors to fly and more suited for outdoors but in an enclosed construction environment there is a potential safety risk for workers and they may even collide with building materials, columns, or overhead cables, (f) legged robots using one or multiple legs inspired from terrestrial mammals to move but maintaining balance while moving is a major challenge for legged robots, (g) hybrid robots using a combination of locomotion systems e.g. a leg-wheel robot.

2.3 Versatility of Legged Robots

Legged robots increase the field of different accessible terrains and are more versatile than wheeled robots providing good potential to traverse construction sites. Mostly used and developed legged robots are two-legged robots, biped or humanoid robots, four-legged robots or quadruped, and six-legged robots or hexapod. Legged robots have become an important research direction in the field of robotics in recent years. Numerous legged robots have been developed by companies, institutions, and individuals including NASA, MIT, IIT, ETH, Boston Dynamics, Ghost Robotics, ANYbotics, etc. But due to high technological complexity to build and control such robots there are only few teams who have developed legged robots that can be applied outside laboratory settings. Multi-legged robots can achieve better locomotion performance in terms of speed, efficiency of movement, and obstacle avoidance. Legged robots have better adaptability on rough terrains, better mobility and flexibility because of many Degrees of Freedom (DoF) of the legs and can move their body independent of underlying terrain making them a good potential for traversing construction environments. [5]

2.4 Quadruped robots

Quadruped robots have a wider range of accessible terrains and are more versatile than wheeled robots, including better mobility, stability, and flexibility due to having more Degrees of Freedom (DoF) per leg. Quadruped robots can traverse uneven terrains, move up and down stairs, and walk over small obstacles, making them more suitable for the construction environment. Quadruped robots can assist humans on job sites in construction inspection and monitoring, but they also require periodic human intervention on unstructured and dynamically changing construction sites. [2]

2.5 Case Studies of Spot in construction industries

2.5.1 Critical Infrastructure: Spot Inspects Hamburg Bridge [6]

Hamburg Port Authority (HPA) not only keeps Germany's largest seaport and Europe's largest rail port running but also looks after real estate and infrastructure in the Port of Hamburg with its approximately 1,800 employees. This includes the many

bridges in the port of Hamburg, including the striking Köhlbrand Bridge, which is about 3.8 km long. The second longest road bridge in Germany, it has been the most important east-west link in the Port of Hamburg for over 50 years. Around 38,000 vehicles cross the bridge every day. With such heavy use and increasing fatigue damage, condition-based maintenance must be carried out with the utmost care. Since 2022, the agile mobile robot Spot® from Boston Dynamics has been helping.

2.5.2 Turner Construction Uses Ground Robots to Cut Inspection Time by Over 95 Percent [7]

Turner Construction is a US-based international construction services company with over 10,000 employees. It is renowned worldwide for its innovative use of technology and completes approximately \$12 billion of construction projects each year. One of the biggest construction users of the DroneDeploy platform, Turner recently decided to add ground robotics to its arsenal of tools for building large-scale data centres across the US. Turner now uses DroneDeploy's Robotics solution to perform autonomous data capture and 360° virtual walkthroughs of its construction sites daily. By sending robots to perform inspections overnight, Turner can accurately, consistently and repeatedly capture progress on some of its biggest and most complex projects—which currently include the construction of large data halls.

2.5.3 RATP: Inspections after dark [8]

The RATP Group (Autonomous Parisian Transportation Administration), is a French state-owned public transport operator and maintainer for the Greater Paris area. With thousands of civil works to inspect each year, the company has turned to mobile robotics to inspect hard-to-reach and hazardous areas to keep employees out of harm's way. Named Perceval by the company, Spot is now fully part of the inspection process.

2.5.4 Virginia Tech [9]

In recent years, robots have achieved a level of mobility that has made them useful on actual job sites in industries like manufacturing and construction. But commercial success doesn't mean that research and development efforts stop, of course. Rather, researchers are following robots into the field, tracking their progress, and helping to identify new opportunities to leverage the emerging technology within a wide range of industry sectors.

Kereshmeh Afsari, an assistant professor in the Myers-Lawson School of Construction at Virginia Tech, along with her team of graduate students and undergraduate researchers and Professor Walid Thabet, were among the first researchers to field test Spot, the quadruped robot from Boston Dynamics. Afsari partnered with Procon Consulting to bring Spot to three of the university's construction projects, and her team spent months testing the robot's inspection capabilities in real-world conditions. Getting out of the lab and onto actual construction sites, Afsari says,

is key for pushing the robotics field beyond the realm of theory and producing practical insights from data generated in real-world environments.

2.5.5 Acciona [10]

Carlos Crespo, head of Acciona's robotics and artificial intelligence group, spends a lot of his day considering the hazards that limit the amount of time humans can spend gathering data from active infrastructure tunnel construction sites. First, there are the explosives that crews use in sequential excavation. Then, after the explosives go off, the tunnel is filled for a period with hazardous gas. Not to mention the dust. And the debris. And the heavy machinery moving about, clearing the way for the next round of explosives. Finally, human workers must wait for the shotcrete that supports the tunnel to dry before they can enter and conduct scans of the site.

This scenario – technical work that needs to be done under potentially dangerous circumstances – is seemingly tailor-made for automation. But the rugged conditions of Acciona's job sites demanded a robotics platform that could handle difficult terrain, operate in tight spaces, carry significant weight, work autonomously, and cover large distances without needing to be recharged.

Spot, the quadruped robot from Boston Dynamics, was a perfect fit.

2.5.6 Pomerleau [11]

Pomerleau saves approximately 20 hours per week by integrating Spot with HoloBuilder's site documentation software on their 500,000 sq. ft. project. It has become crucial for Pomerleau's business strategy to integrate innovative technologies and streamlined workflows that enhance employee productivity by reducing the resources required to perform repetitive and often unfavourable tasks. The highest concentration of these tasks involves capturing data to monitor the progression of a project, particularly when obtaining data from restricted or dangerous areas. Pomerleau would like to use automated systems like Spot to complete these tasks and free up time and resources for employees to focus on higher-level tasks.

Pomerleau equipped Spot with a 360° camera to allow it to capture completed work on the jobsite. Spot's image capture capability is currently linked to the HoloBuilder platform. This integration allows the site manager to compare weekly or even daily images to track the site's progress. These images can be compared to the 3D model to visually identify any differences, aiding in the coordination of on-site subcontractors and tradespeople.

2.5.7 Brasfield & Gorrie [12]

General contractor Brasfield & Gorrie integrates Spot with DroneDeploy for automated and centralized site documentation. Like many general contractors,

Alabama-based Brasfield & Gorrie is familiar with the operational challenges of collecting 360° documentation to monitor progress on their jobsites.

Traditionally, the company has outsourced the work to third party documentation services and found that, while valuable, the imagery could be costly to gather. When cloud-based construction project management apps started to enable on-the-go documentation with mobile devices, Brasfield & Gorrie began transitioning to performing the work in-house on many projects so they could bring it under their own contract. But this inadvertently created another problem: It gave a tedious, time-consuming task to a project manager or engineer with an already full to-do list.

The company found a solution for automation in a customized robotic platform developed in partnership with Boston Dynamics and DroneDeploy.

2.5.8 Foster + Partners [13]

When Foster + Partners brought Spot to a London project site for the first time, construction workers and office personnel temporarily morphed into amateur paparazzi and star-struck fans.

This sort of reaction is, perhaps, understandable. While millions of people have watched Spot traverse tricky terrain in online videos, very few have seen the advanced quadruped robot in real life. But the ARD group at Foster + Partners didn't bring in Spot so they could marvel at a novelty. Rather, the practice put the robot through rigorous, real-world testing to prepare for the day when Spot's appearance at a building site will seem as ordinary as the arrival of a bulldozer or a forklift do today.

2.5.9 Swinerton [14]

The innovation team at Swinerton, a major US-based general contractor, develops new methods and processes to solve construction's oldest problems. Recently they began the process of automating progress tracking on work sites. After a period of development, Swinerton deployed the Spot platform to capture data at an active project in the Bay Area outside of San Francisco. The site spanned four floors, 197,800 square feet, and nearly 300 rooms.

The company estimates this automated process could reduce the time for final payment from 90 days to as little as 10 days and eliminate the myriad downstream problems caused by slow payment processing.

The implementation of Spot has also started to change the culture at Swinerton. It has pushed parts of the company to re-think large parts of their processes—and even refresh their approach to technology on the job site.

3 Methodology

3.1 Theory and Documentation

The first step was reading the documentation provided by Boston Dynamics. The SDK documentation explains [15]:

1. Conceptual documentation:
These documents explain the key abstractions used by the Spot API. It helps to get familiar with the various terms used in the documentation for further development.
2. Python client library:
Applications using the Python library can control Spot and read sensor and health information from Spot. A wide variety of example programs and a QuickStart guide are also included.
3. Payload developer documentation:
Payloads add additional sensing, communication, and control capabilities beyond what the base platform provides. The Payload ICD covers the mechanical, electrical, and software interfaces that Spot supports.
4. SPOT API protocol definition:
This reference guide covers the details of the protocol applications used to communicate to Spot. Application developers who wish to use a language other than Python can implement clients that speak the protocol.
5. Spot SDK Repository:
The GitHub repo where all the Spot SDK code is hosted by Boston Dynamics

Understanding at least the main concepts related to SPOT is very important to further research with it. E.g. Networking, E-Stop, KeepAlive, Lease, Autonomy services. Also, the SDK is maintained by Boston Dynamics and there will be updates in the future as research progresses.

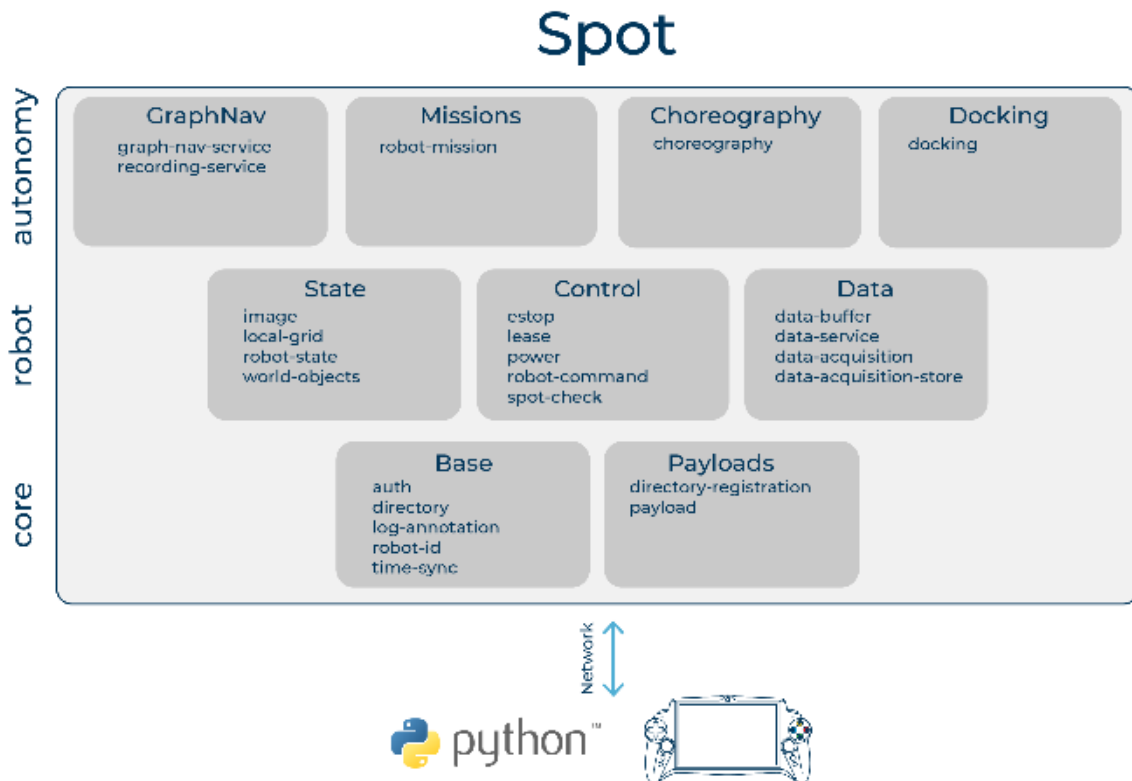


Figure 1: Block Diagram showing three different level of robot's services. Core, robot and autonomy. The Base, Payloads, Control and Data blocks are the necessary services that are a must to connect with SPOT.

3.2 Communication and Networking

SPOT can be connected in both wired and wireless mode, with feasibility of both being an access point or a client in WiFi mode or even via custom communication links. gRPC is the application-level protocol used by most of the SPOT API. [16] The open-source RPC framework developed by google. (0)

The clients and the server are connected using this framework. In our case, the SPOT robot is the server, and the client can be either the tablet or any connected PC. Protocol buffers are used to communicate. Protocol Buffers are a language-neutral, platform-neutral extensible mechanism for serializing structured data. Google's mature open-source mechanism for serializing structured data. The protocol buffers are defined using ".proto" extension definition files. The structure for the data is defined in this file. A protocol buffer compiler (protoc) generates data access classes in your preferred language(s) from your proto definition.

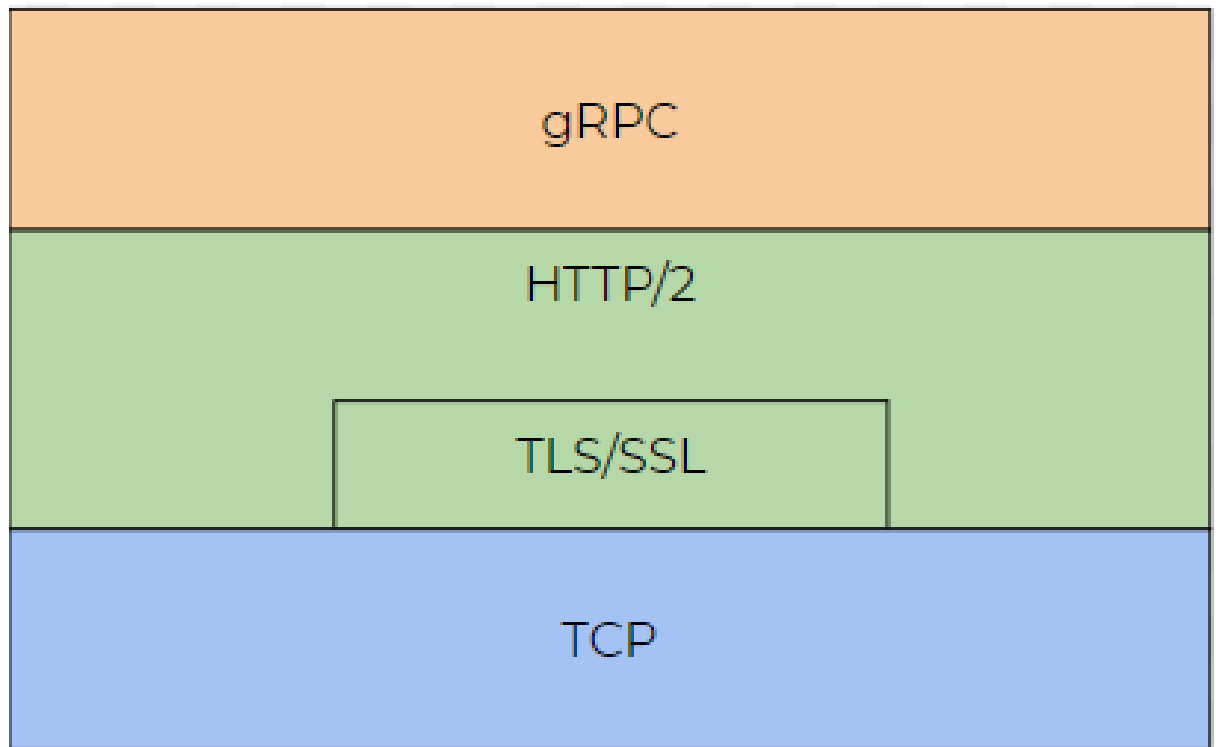


Figure 2: gRPC is built on top of other networking protocols, as shown in the diagram above.

3.3 Hardware

3.3.1 SPOT

SPOT is one of the most versatile robots available in the market capable of carrying heavy load for a longer time than other competitors. (Appendix B) [17]

3.3.1.1 Cameras

SPOT has five built in cameras (fish-eye and depth cameras) to observe the surroundings. Spot exposes these cameras as image sources via the on-robot Image service, enabling multi-camera capture with hardware time-sync when multiple sources are requested in a single RPC.

Table A-1

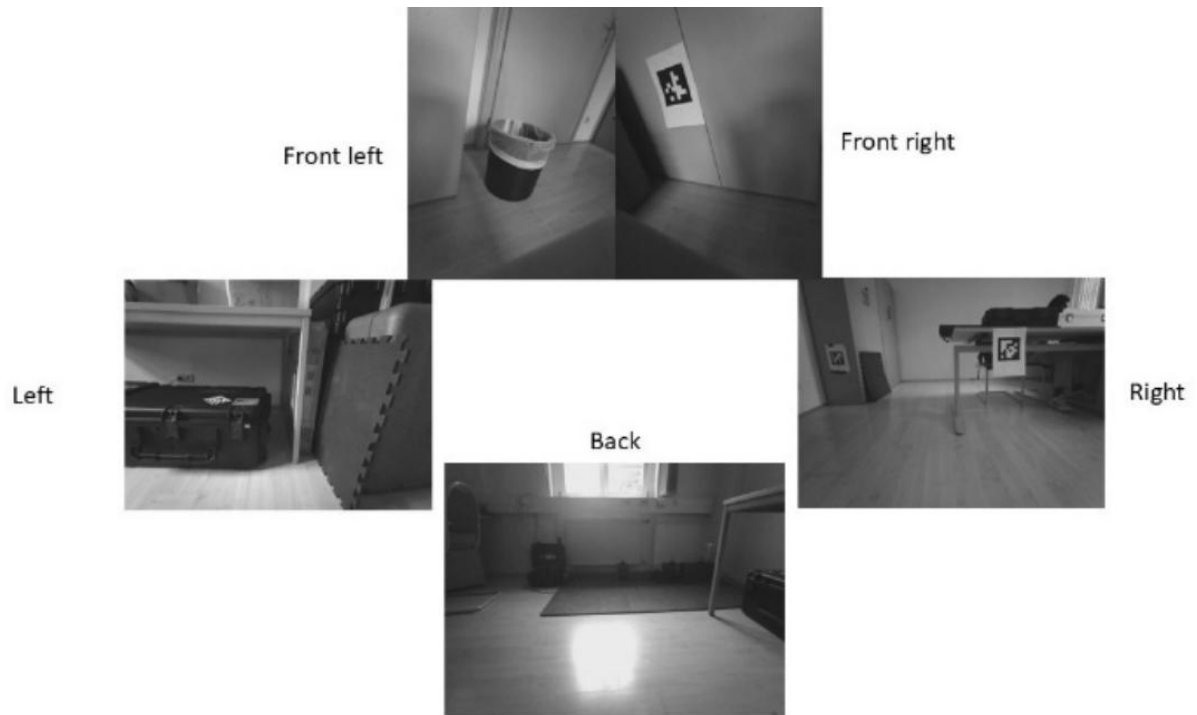


Figure 3: Collective view of all the cameras of SPOT

3.3.2 Core I/O

A secondary PC which enhances both computation and communications available on the Spot platform. [18]



Figure 4: SPOT Core I/O payload

3.3.3 Ricoh Theta Z1 360° camera [19]

The Spot API has a feature to connect with Ricoh Theta (a 360° view camera) either through Core I/O or directly to the client PC.



Figure 5: Ricoh Theta Z1 360° camera, which has the possibility of being used as the payload.

3.3.4 X-Box controller

The Xbox Wireless Controller is the primary game controller for Microsoft's Xbox One and Xbox Series X/S home video game consoles, as well as the official controller for use on Windows-based PCs; it is also compatible with other operating systems such as macOS, Linux, iOS and Android.

In the SPOT SDK, a sample program is given to control Spot's movement using a controller by implementing the XInput API which helps windows PC to communicate with the controller.



Figure 6: X-Box controller (with wired and wireless functionality)

3.3.5 Client System

A PC/Laptop is required to act as a client for communication and programming using the Spot API. At times, the client collects raw data from robot and then analysis is performed on the client system to save Spot from computation overload.

3.3.6 Fiducial tags

The Spot's world object service can recognise QR codes mostly referred to as fiducial tags of the belonging to AprilTags in the Tag36h11 set. The default Image size: 146 mm square. Printed on white non-glossy U.S. letter-size sheets (preferably rigid). For 500-series fiducials used with a Spot Dock, the outer boundary of the fiducial sheet should be 200.8 mm wide x 215.5 mm high. For wall-mounted fiducials, the ideal position is 45 to 60 cm above the ground. The range of detection for a normal size fiducial is 3 meters. However, the size of the fiducial can be adjusted from the SPOT Admin Console.

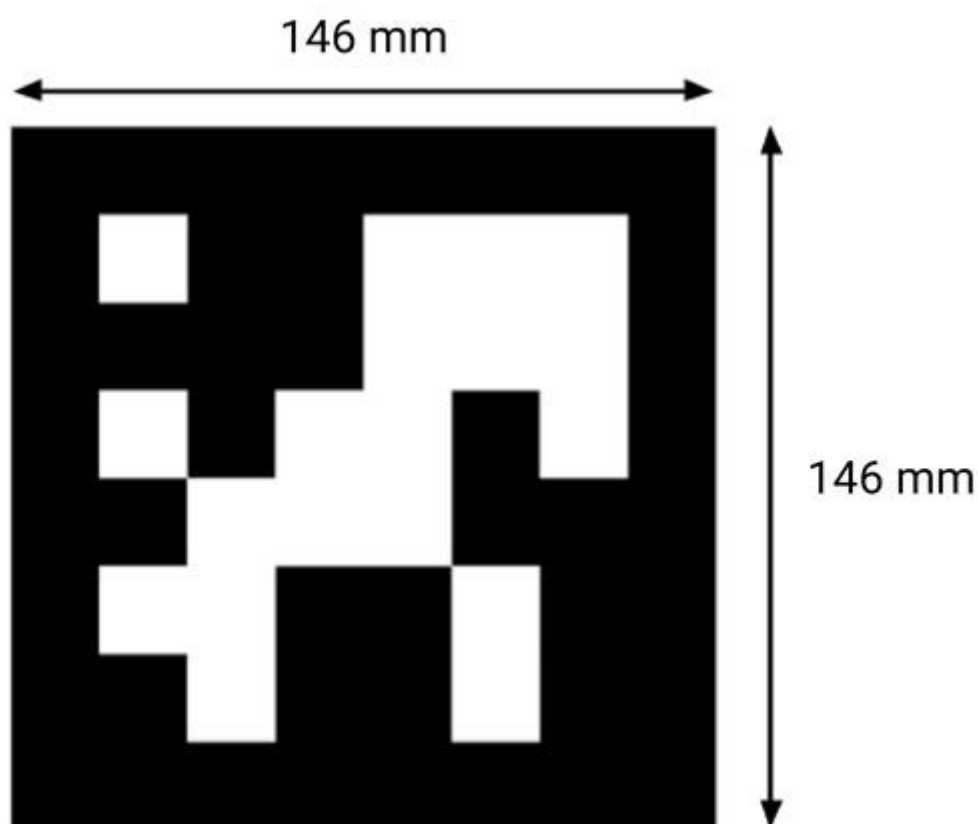


Figure 7: Fiducial tag with dimensions

3.4 Software

3.4.1 Microsoft Visual Studio Code (VS Code)

Visual Studio Code is an integrated development environment developed by Microsoft for Windows, Linux, macOS and web browsers. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded version control with Git. Although, VS Code is not a full-fledged IDE, it's a lightweight code editor which can be extended with many features which makes it

equal to some IDE-like functionalities, it proved to be a useful tool for version control and to write and edit code. Users can change the theme, keyboard shortcuts and preferences, as well as install extensions that add functionality.

3.4.2 GitHub

GitHub is a proprietary developer platform that allows developers to create, store, manage, and share their code. It uses Git to provide distributed version control and GitHub itself provides access control, bug tracking, software feature requests, task management, continuous integration, and wikis for every project. It is commonly used to host open-source software development projects. As of January 2023, GitHub reported having over 100 million developers and more than 420 million repositories, including at least 28 million public repositories. It is the world's largest source code host as of June 2023. Over five billion developer contributions were made to more than 500 million open-source projects in 2024.

3.4.3 Spot SDK

The SDK consists of the RPC protobuf file required for communication along with high level programs and example programs that can be used to perform some action using Spot. In practice, it helps to connect to Spot's gRPC endpoints, authenticate, time-synchronize and then acquire a lease before issuing and command. – these guardrails ensure only one controller owns the robot at a time. The example programs in the SDK, show safe patterns for teleop, image capture, data acquisition, docking, mobility control including how to set velocity limits and other parameters.

Critical safety features are built in an E-Stop interface for immediate halt, capability checks to warn the user if unsupported features are called. Service APIs like RobotState, Image, Data Acquisition, RobotCommand, TimeSync and Docking are consistently designed and well documented. The API is very powerful and well maintained by Boston Dynamics where users can experiment with the functionalities of SPOT without causing major damage. Version used for project: 5.0.1

3.4.4 Python Programming language

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Python is dynamically type-checked and garbage-collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming. Python is a versatile and easy-to-learn language that lets you work quickly and integrate systems more effectively. Version used for project: Python 3.10

4 Implementation and Test cases

4.1 System Architecture

High-level architecture of the Spot program. The operator can issue **manual commands at any time** (Manual Override Module) while fiducial follow executes fiducial-triggered tasks. Perception feeds fiducial detection; the Task Manager performs actions (e.g., video streaming, movement, digital signal to the client). Safety (E-Stop, lease, auth) supervises all actuation. Telemetry, images, and signals flow to the client system and optional storage/BIM server.

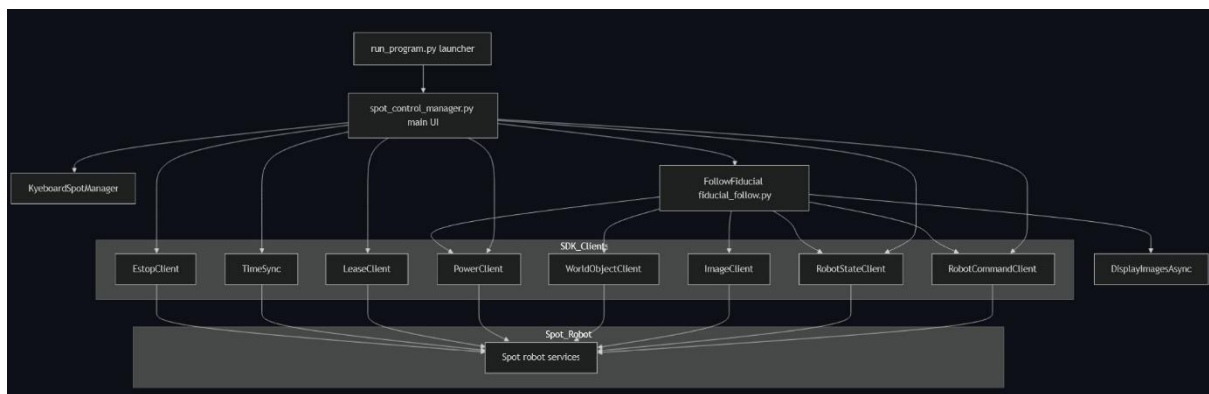


Figure 8: High level System Architecture of the spot program

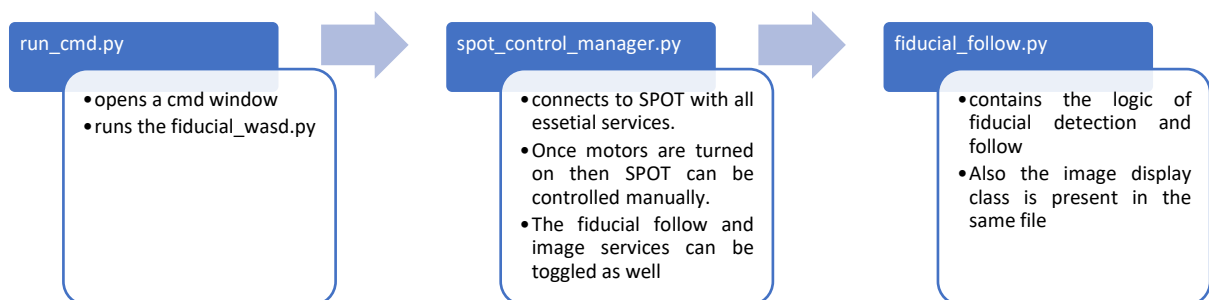


Figure 9: Code Organization. Implemented by considering the Software engineering principles.

4.2 Python program [20]

In the entire program the, the obstacle detection is kept on for the safety of the robot and the operator as well. This is done in both the manual and fiducial follow program files.

4.2.1 Run_cmd.py

This part of the code is used to run the main program of “spot_control_manager.py”. It can be customised to run a different program at the same path and has info on the Spot’s IP address.

```
1  import os
2  import subprocess
3
4  """Script to run program on a robot using the Spot SDK."""
5
6  # Configuration
7  program_directory = os.path.dirname(os.path.abspath(__file__))
8  program_name = "spot_control_manager.py"
9  robot_ip = "192.168.80.3"
10
11 # Change to the directory where the program is located
12 os.chdir(program_directory)
13
14 # Run the program
15 subprocess.run(['cmd', '/c', 'start', 'cmd', '/k', 'python3', program_name, robot_ip])
```

Figure 10: run_cmd.py. To initiate/run the program

4.2.2 Spot_control_manager.py

The program with main services running and establishing a single point of communication. The program consists of classes:

- ExitCheck: To safely exit the program by sitting down Spot and then turning motor power OFF.
- CurserHandler: Class for managing the logs and errors visible on the interface.
- AsyncRobotState: Use to get the status/state of the Robot. Very important to manage E-Stop and Lease. Knowing the state of the robot is essential for the program to find out any errors that can occur during operation or obstacles faced by Spot during navigation.
- KeyBoardSpotManager: The main parent class where the all the functionalities are called. Hence the name “manager”. The manual keyboard input is received in this class.

Keys	Feature	Explanation
TAB	Quit	Quits the program immediately
T	Time-sync	Time syncs the client and robot time
SPACE	E-Stop	Toggles E-Stop
p	Power	Toggles motor power (Motor power can only be toggled if E-Stop is not activated)
f	Stand	Spot stands
r	Self-right	Spot turns back up from battery change position
wasd	Directional strafing	For movement in four directions
v	Sit	Spot sits
b	Battery-change	Spot tilts and rests on two legs, allowing battery to be changed easily. Note: Spot turns off while exchanging batteries
qe	Turning	For turning the Spot
i	Camera view	Toggles the video stream
ESC	Stop	Stops the robot immediately at place.
o	Toggle fiducial follow	Spot starts to detect fiducials and can follow the master tag
l	Return/Acquire lease	Toggles lease
m	Next fiducial code	Cycle through code list in forward direction
n	Previous fiducial code	Cycle through code list in backward direction
x	Toggle follow current fiducial ID	A toggle switch to follow/ unfollow selected fiducial

Table 1: Key bindings for manual input

Also, the class is responsible to establish an end-to-end valid RPC connection by acquiring Lease, E-Stop control, TimeSync and motor control.

- There are other universal methods visible that are required to ensure proper visibility of the interface and logging of data.

```

1  # Copyright (c) 2023 Boston Dynamics, Inc. All rights reserved.
2  #
3  # Downloading, reproducing, distributing or otherwise using the SDK Software
4  # is subject to the terms and conditions of the Boston Dynamics Software
5  # Development Kit License (20191101-BDSDK-SL).
6
7  import curses
8  import io
9  import logging
10 import math
11 import os
12 import signal
13 import sys
14 import threading
15 import time
16 from collections import OrderedDict
17 import locale
18 locale.setlocale(locale.LC_ALL, '') # better width handling for unicode/wide chars
19 from types import SimpleNamespace
20
21 import bosdyn.api.basic_command_pb2 as basic_command_pb2
22 import bosdyn.api.spot.robot_command_pb2 as spot_command_pb2
23 import bosdyn.api.robot_state_pb2 as robot_state_proto
24 import bosdyn.api.power_pb2 as PowerServiceProto
25 import bosdyn
26 import bosdyn.client.util
27 from bosdyn.client.estop import EstopClient, EstopEndpoint, EstopKeepAlive
28 from bosdyn.client.exceptions import ResponseError, RpcError
29 from bosdyn.client.frame_helpers import ODOM_FRAME_NAME
30 from bosdyn.client.robot_command import RobotCommandBuilder, RobotCommandClient
31 from bosdyn.client.robot_state import RobotStateClient
32 from bosdyn.client.lease import LeaseClient, LeaseKeepAlive
33 from bosdyn.client.power import PowerClient
34 from bosdyn.client.sdk import create_standard_sdk
35 from bosdyn.client.time_sync import TimeSyncError
36 from bosdyn.client.async_tasks import AsyncGRPCTask, AsyncPeriodicQuery, AsyncTasks
37 from bosdyn.client.lease import Error as LeaseBaseError
38 from bosdyn.util import duration_str, format_metric, secs_to_hms
39
40 from fiducial_follow import FollowFiducial, DisplayImagesAsync
41
42 VELOCITY_BASE_SPEED = 0.5 # m/s
43 VELOCITY_BASE_ANGULAR = 0.8 # rad/sec
44 VELOCITY_CMD_DURATION = 0.6 # seconds
45 COMMAND_INPUT_RATE = 0.1
46
47 #pylint: disable=no-member
48 LOGGER = logging.getLogger("spot_control_manager")
49
50 > def _safe_addstr(win, y, x, s): ...
69
70 > def _ensure_min_size(stdscr, min_rows=17, min_cols=600): ...
84
85 > def _try_resize(stdscr, rows=40, cols=300): ...
93
94 > class ExitCheck(object): ...
119
120
121 > class CursesHandler(logging.Handler): ...
132
133
134 > class AsyncRobotState(AsyncPeriodicQuery): ...
143
144 > class KeyboardSpotManager(object): ...
640
641
642 > def _setup_logging(verbose): ...
668
669 > def main(): ...
721
722
723 > if __name__ == '__main__': Abhishek Patil, last month • working fiducial follow and wasd code _ ...

```

Figure 11: spot_control_manager.py

4.2.3 Fiducial_follow.py

The program file containing the fiducial follow class and the Video streaming class (The video stream is achieved by streaming pictures through gRPC).

- FollowFiducial: Consists of logic where the fiducial is detected and then the robot is aligned to the fiducial with the proper heading. There are even different tasks assigned for specific tag Ids in the same class.

Master fiducial: The Spot will follow this fiducial once it has been detected and give priority to it over any other fiducial. (In our case tag id 5). Also, a dog sound is played at the client PC when Spot is following this tag.

Reaction fiducials: The Spot can react to fiducial (in our case tag id 10) with a JOG movement while approaching them depicting a playful nature of a dog.

Normal fiducials: These are fiducials with no assigned special tasks and Spot can detect and follow them if such input is given by the user.

- All the functionalities are called from the KeyboardSpotManager class by key press inputs from the user as explained before.
- The video streaming class is also in the same file as there is a dependency with the fiducial follow. The video stream however only shares snips of images for the user to see the view of Spot. Also, the images are converted before sending over the network to not overload the connection with heavy data.

```

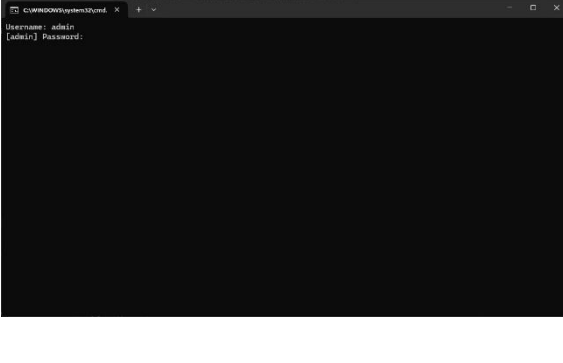
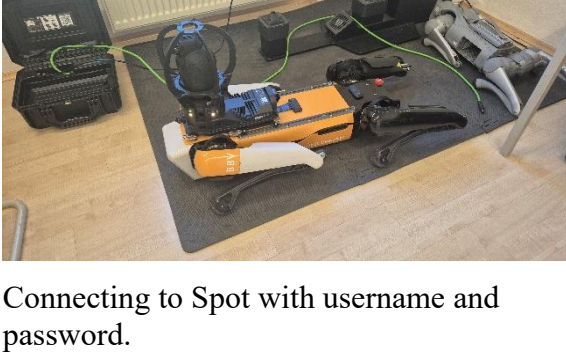
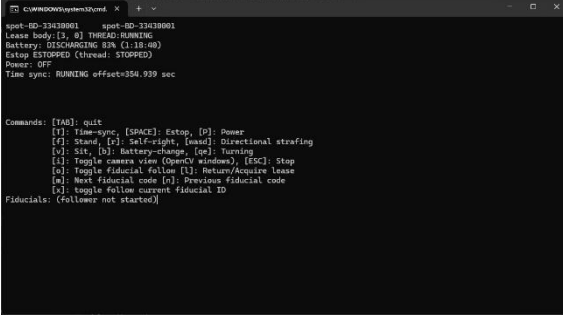
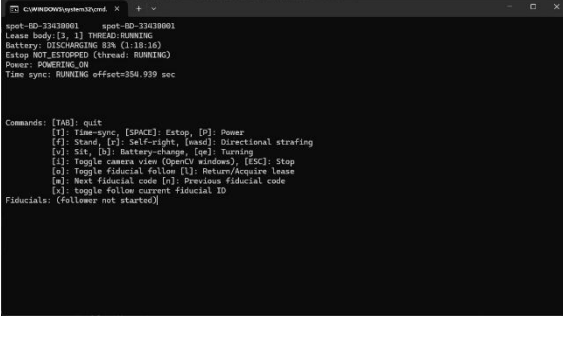
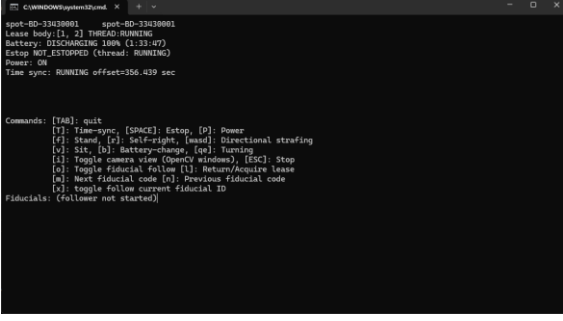
1  # Copyright (c) 2023 Boston Dynamics, Inc. All rights reserved.
2  #
3  # Downloading, reproducing, distributing or otherwise using the SDK Software
4  # is subject to the terms and conditions of the Boston Dynamics Software
5  # Development Kit License (20191101-BDSDK-SL).
6
7  """ Detect and follow fiducial tags. """
8  import logging
9  import math
10 import signal
11 import sys
12 import threading
13 import time
14 from sys import platform
15 import winsound
16
17 import cv2
18 import numpy as np
19 from PIL import Image
20
21 import bosdyn.client
22 import bosdyn.client.util
23 from bosdyn import geometry
24 from bosdyn.api import geometry_pb2, image_pb2, trajectory_pb2, world_object_pb2
25 from bosdyn.api.geometry_pb2 import SE2Velocity, SE2VelocityLimit, Vec2
26 from bosdyn.api.spot import robot_command_pb2 as spot_command_pb2
27 from bosdyn.client import ResponseError, RpcError, create_standard_sdk
28 from bosdyn.client.frame_helpers import (BODY_FRAME_NAME, VISION_FRAME_NAME, get_a_tform_b,
29                                          get_vision_tform_body)
30 from bosdyn.client.image import ImageClient, build_image_request
31 from bosdyn.client.lease import LeaseClient
32 from bosdyn.client.math_helpers import Quat, SE3Pose
33 from bosdyn.client.power import PowerClient
34 from bosdyn.client.robot_command import RobotCommandBuilder, RobotCommandClient, blocking_stand
35 from bosdyn.client.robot_id import RobotIdClient, version_tuple
36 from bosdyn.client.robot_state import RobotStateClient
37 from bosdyn.client.world_object import WorldObjectClient
38
39 #pylint: disable=no-member
40 LOGGER = logging.getLogger("fiducial_follow")
41
42 # Use this length to make sure we're commanding the head of the robot
43 # to a position instead of the center.
44 BODY_LENGTH = 1.1
45
46 You, 2 days ago | 3 authors (NirajLadhe and others)
47 > class FollowFiducial(object): ...
419
420 NirajLadhe, last month | 1 author (NirajLadhe)
420 > class DisplayImagesAsync(object): ...

```

Figure 12: *fiducial_follow.py*

4.3 Working of the Program

Below is a depiction of the flow of basic function of the program

1	 <pre> C:\WINDOWS\system32\cmd.exe Username: admin [admin] Password: </pre>	 Connecting to Spot with username and password.
2	 <pre> spot-ID-234230001 spot-ID-234230001 Lease body:[3, 0] THREAD-RUNNING Battery: DISCHARGING 83% (1:18:40) Estop: ESTOPPED (thread: STOPPED) Power: OFF Time sync: RUNNING offset=354.939 sec Commands: [TAB]: quit [1]: Time-sync, [SPACE]: Estop, [D]: Power [F]: Stand, [r]: Self-right, [msd]: Directional strafing [s]: Sit, [b]: Battery-charge, [ep]: Turning [1]: Toggle camera view (OpenCV windows), [ESC]: Stop [a]: Toggle fiducial follow [1]: Return/acquire lease [n]: Next fiducial code [n]: Previous fiducial code [x]: toggle follow current fiducial ID Fiducials: (follower not started) </pre>	 Checking lease connection and Time sync.
3	 <pre> spot-ID-234230001 spot-ID-234230001 Lease body:[3, 1] THREAD-RUNNING Battery: DISCHARGING 83% (1:18:40) Estop: NOT-ESTOPPED (thread: RUNNING) Power: POWERING_ON Time sync: RUNNING offset=354.939 sec Commands: [TAB]: quit [1]: Time-sync, [SPACE]: Estop, [D]: Power [F]: Stand, [r]: Self-right, [msd]: Directional strafing [s]: Sit, [b]: Battery-charge, [ep]: Turning [1]: Toggle camera view (OpenCV windows), [ESC]: Stop [a]: Toggle fiducial follow [1]: Return/acquire lease [n]: Next fiducial code [n]: Previous fiducial code [x]: toggle follow current fiducial ID Fiducials: (follower not started) </pre>	 Disengaging E-Stop and turning Motor power ON.
4	 <pre> spot-ID-234230001 spot-ID-234230001 Lease body:[3, 2] THREAD-RUNNING Battery: DISCHARGING 100% (1:33:47) Estop: NOT-ESTOPPED (thread: RUNNING) Power: ON Time sync: RUNNING offset=356.439 sec Commands: [TAB]: quit [1]: Time-sync, [SPACE]: Estop, [D]: Power [F]: Stand, [r]: Self-right, [msd]: Directional strafing [s]: Sit, [b]: Battery-charge, [ep]: Turning [1]: Toggle camera view (OpenCV windows), [ESC]: Stop [a]: Toggle fiducial follow [1]: Return/acquire lease [n]: Next fiducial code [n]: Previous fiducial code [x]: toggle follow current fiducial ID Fiducials: (follower not started) </pre>	 Manually standing up the robot.

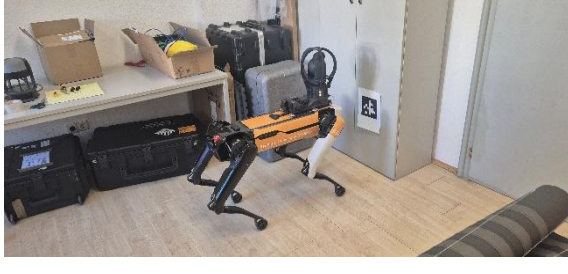
5	<pre> spot-RO-33430001 spot-RO-33430001 Lease body: [5, 3] THREAD: RUNNING Battery: DISCHARGING 70% (112.62) Estop NOT_STOPPED (thread: RUNNING) Power: ON Time sync: RUNNING offset=354.909 sec Fiducial follow: STARTED Camera viewer: STOPPED Commands: [F40]: quit [1]: Time-sync, [SPACE]: Estop, [D]: Power [F]: Stand, [r]: Self-right, [wasd]: Directional strafing [S]: Sit, [D]: Battery-change, [e]: Turning [I]: Toggle camera view (OpenCV windows), [ESC]: Stop [a]: Toggle fiducial follow [1]: Return/Require lease [m]: Next fiducial code [n]: Previous fiducial code [D]: toggle follow current fiducial ID Fiducials visible: [7, 200] Selected: - Follow selected: OFF Fiducial follow thread: RUNNING Priority: Tag 5 overrides any selection when visible. </pre>	 Scanning for fiducials
6	<pre> spot-RO-33430001 spot-RO-33430001 Lease body: [5, 5] THREAD: RUNNING Battery: DISCHARGING 70% (111.60) Estop NOT_STOPPED (thread: RUNNING) Power: ON Time sync: RUNNING offset=354.909 sec Follow selected: ON Selected fiducial: 7 Fiducial follow: STARTED Commands: [F40]: quit [1]: Time-sync, [SPACE]: Estop, [D]: Power [F]: Stand, [r]: Self-right, [wasd]: Directional strafing [S]: Sit, [D]: Battery-change, [e]: Turning [I]: Toggle camera view (OpenCV windows), [ESC]: Stop [a]: Toggle fiducial follow [1]: Return/Require lease [m]: Next fiducial code [n]: Previous fiducial code [D]: toggle follow current fiducial ID Fiducials visible: [7, 200] Selected: 7 Follow selected: ON Fiducial follow thread: RUNNING Priority: Tag 5 overrides any selection when visible. </pre> Selecting and walking towards the selected fiducial	 
7	<pre> spot-RO-33430001 spot-RO-33430001 Lease body: [5, 3] THREAD: RUNNING Battery: DISCHARGING 70% (111.60) Estop NOT_STOPPED (thread: RUNNING) Power: ON Time sync: RUNNING offset=354.909 sec Selected fiducial: 7 Fiducial follow: STARTED Camera viewer: STOPPED Commands: [F40]: quit [1]: Time-sync, [SPACE]: Estop, [D]: Power [F]: Stand, [r]: Self-right, [wasd]: Directional strafing [S]: Sit, [D]: Battery-change, [e]: Turning [I]: Toggle camera view (OpenCV windows), [ESC]: Stop [a]: Toggle fiducial follow [1]: Return/Require lease [m]: Next fiducial code [n]: Previous fiducial code [D]: toggle follow current fiducial ID Fiducials visible: [7, 200] Selected: 7 Follow selected: OFF Fiducial follow thread: RUNNING Priority: Tag 5 overrides any selection when visible. </pre> Manually bringing back Spot and starting to power down procedure in the reverse order as done during powering up.	



Table 2: basic function flow of the program

Note: The world object service keeps the info of the tag even after the tag is removed from the Spot vision for about 7~15 sec. This is done to not lose the Fiducial info while navigating towards one and accidentally, losing data about the fiducial as it vanishing from the visible area.

4.4 Test cases

Additional test cases were also researched in the duration of the project.

4.4.1 Detecting fiducials using at the client end using AprilTag Library

The first trail for detecting fiducial tag was implemented by using the open source AprilTag Library. In this case only the camera feed was received from the Spot as raw data. Then the client PC ran an algorithm to detect fiducial in the incoming stream. After that the distance and the heading were calculated with the help of the library. And then the movement command was given to Spot accordingly. The drawbacks were, improper heading, doggy movement. Laggy video stream due to huge amount of data being sent through the network for analysis. Hence, it was better to stick with the in-built “world object detection” of Spot for identifying the tags and the robot “get_vision_tform_body” service is used to transform the coordinates from the vision frame to the body frame. This resulted in highly accurate positioning and less load on the client PC and the network.

4.4.2 Possible to descend stairs in manual mode, but tricky in fiducial follow

Spot has an inbuilt flag which detects the stairs in its surroundings. By default, this is true. While ascending the stairs, the heading of the robot and the direction in which it can ascend is the same (forward direction). Hence Spot can ascend stairs in fiducial follow mode as well. Contrary to this, Spot can only descend the stairs with hind legs first. i.e. in backward direction. A preliminary code was tested with the Spot where the heading used to dynamically change to back_fish_eye. When stairs were detected. However, if the robot stops in the middle of the stairs. It finds difficult to acknowledge that it is still positioned on the stairs. The robot has been observed to change the heading back to front direction in midway. Hence, more work and time is needed to come up with a robust code that can handle such complex situations for autonomous fiducial follow. On the other hand, during manual control the operator can simply turn the robot around and descend the stairs if needed. Hence, proving the benefit of having manual control along with autonomous behaviour.

4.4.3 Usage of Core I/O for more advance interaction with surroundings

Core I/O can increase the computational power of Spot hence allowing more complex sensors to be mounted with Spot where the data is first partially stored and processed in Core I/O before sending it to the client. The application of Core I/O is with heavy payload like scanners and high-definition cameras to handle the huge data without interfering with the core features of Spot. Unfortunately, due to technical problems with the Core I/O further research was inhibited and only theoretical research was done. During research, it was found that custom programs can be uploaded on the

Core I/O and deployed with the help of Docker system. This can help to perform high computational task on the Spot body itself withing Core I/O keeping the communication between client and Spot clear. This can help in features such as image/ object recognition, machine learning, etc. [15]

4.4.4 Testing with other equipment such as Ricoh Theta Z1 360° camera and Controllers

While using Ricoh Theta with Spot is a common example as there have been applications before in case-studies where Spot was mounted with high-definition cameras for inspection. It made us curious to understand how a camera can be integrated with the Spot. We were able to integrate it with the client PC over the same network as Spot.

Also, the controller was tested to control Spot manually with the help of “XInput” library for windows PC. Both the tests were success, but due to time limitation of the project, this has been kept open for further work.

4.5 Results and Discussion

The implemented program is robust and capable of navigating with the help of fiducial ID detection. Successful distinction between different IDs is possible which can also help to make Spot react to certain tags. The control manager ensures a single and secure connection is established to avoid any gRPC errors and timeouts.

The Spot is always interacting with the surrounding, scanning for fiducials and responding what is visible to the operator. Then decisions can be made accordingly for navigation. The camera feed also provides a safety feature for the operator to look at the world through Spot eyes. When the need is felt, the operator can guide Spot through difficult-to-navigate terrain. As the feedback loop is still not as fast as an actual person. But the ability to work in hazardous environment and carry decent amount of load is what appeals for the usage of Spot.

The additional test cases performed in course of the project compliment the base of the program and with additional payloads, it can be brought closer to a real interaction between an organism and the surrounding.

5 Conclusion and Future Work

5.1 Conclusion

As depicted in below diagrams, one can find advantages of having manual control alongside autonomous behaviour.

To implement changes in the auto walk. The entire path needs to be re-recorded, or a first manual run is required. And no deviation from the path is allowed during the autonomous navigation. Hence one must think before assigning the path to the robot for inspection for data collection. And after regular time interval it must be reviewed to check that the current path is the optimal path required for the project. Although it does not have major impact on completed sites where the point of inspection is almost static.

An occasionally failure can be detected beyond the usual inspection route. Then it is much faster and easier with fiducial tags to change the route of the Spot and assign new tasks and responsibilities.

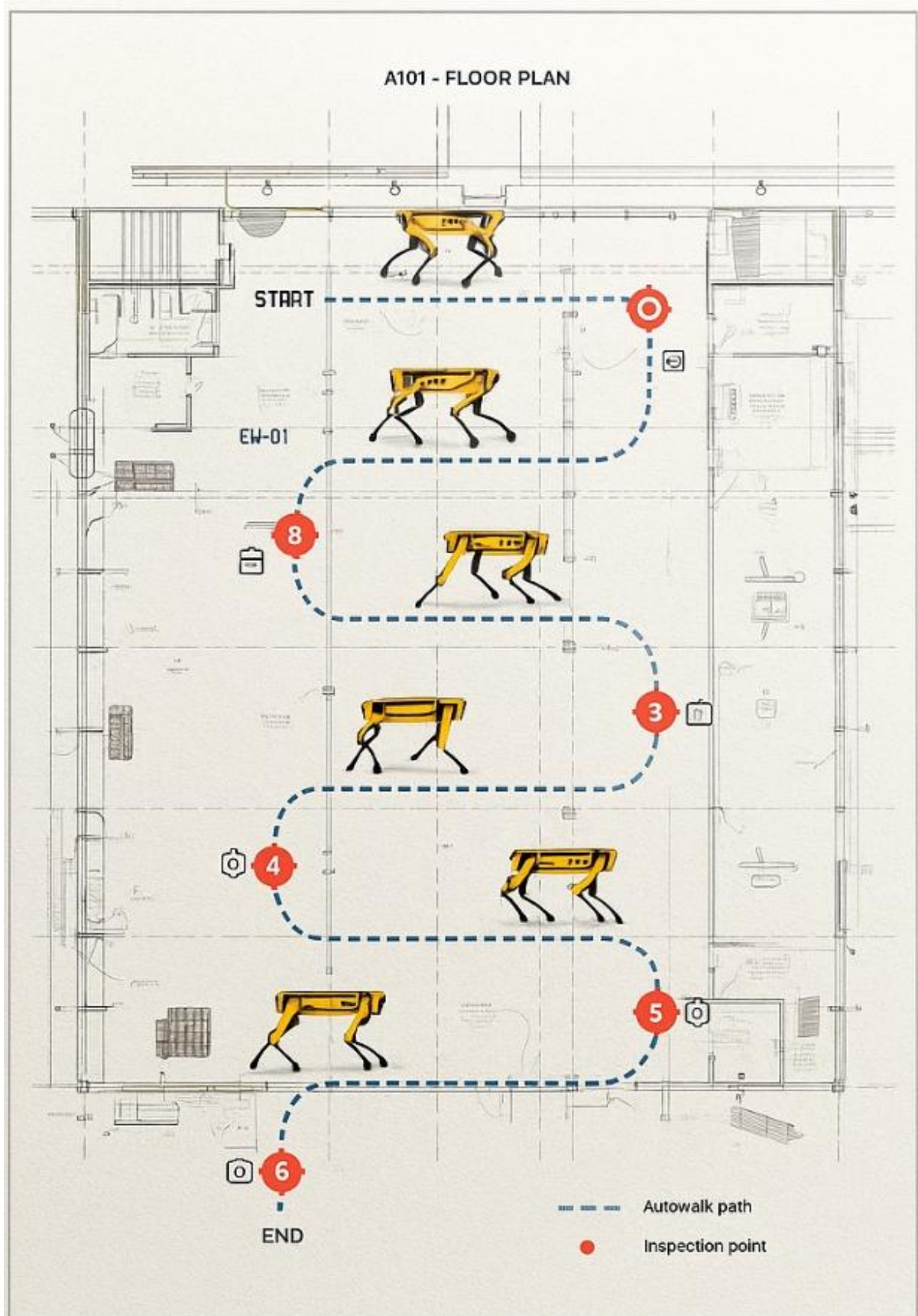


Figure 13: Autowalk/ Most common autonomous applications, image source: Sora

With fiducial detection, even the path can be varied by changing the position of the tags as per the changes in the environment of the construction site without any manual intervention with the movement of the Spot. Making the behaviour more interactive with the environment.

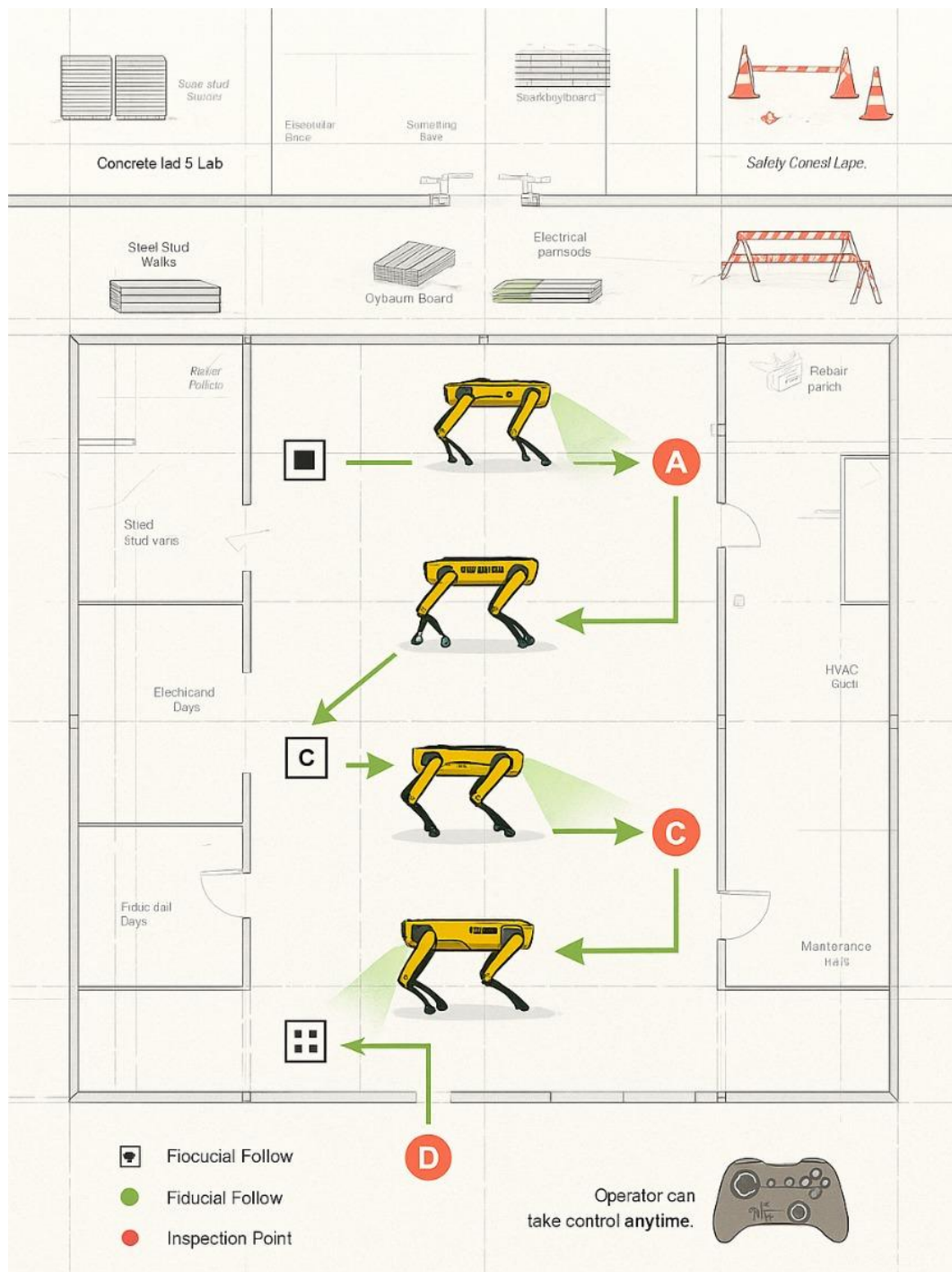


Figure 14: Combination of autonomous fiducial follow and manual control, image source: Sora

5.2 Future Work

With more advance sensors and computational power, higher level of automation can be achieved without compromising with the Human oversight. Closing the gap between man and machine for a better and smoother workflow, enhancing safety and productivity.

References

- [1] Navon, R., Sacks, R.: ‘Assessing research issues in Automated Project Performance Control (APPC)’, *Automation in Construction*, 2007, **16**, (4), pp. 474–484
- [2] Halder, S., Afsari, K., Chiou, E., Patrick, R., Hamed, K.A.: ‘Construction inspection & monitoring with quadruped robots in future human-robot teaming: A preliminary study’, *Journal of Building Engineering*, 2023, **65**, p. 105814
- [3] Kopsida, M., Brilakis, I., Vela, P.A., eds.: ‘e: A Review of Automated Construction Progress Monitoring and Inspection Methods’ (2015)
- [4] Halder, S., Afsari, K., Serdakowski, J., DeVito, S.: ‘A Methodology for BIM-enabled Automated Reality Capture in Construction Inspection with Quadruped Robots’, *ISARC Proceedings*, 2021, **2021 Proceedings of the 38th ISARC, Dubai, UAE**, pp. 17–24
- [5] Afsari, K., Halder, S., Ensafi, M., DeVito, S., Serdakowski, J.: ‘Fundamentals and Prospects of Four-Legged Robot Application in Construction Progress Monitoring’. ASC 2021. 5th Annual Associated Schools of Construction International Conference, California State University, Chico, 274-263
- [6] ‘Critical Infrastructure: Spot Inspects Hamburg Bridge | Boston Dynamics’, <https://bostondynamics.com/case-studies/critical-infrastructure-spot-inspects-hamburg-bridge/>, accessed 28-09-25
- [7] ‘Turner Uses Robots to Cut Inspection Time Over 95 Percent | Boston Dynamics’, <https://bostondynamics.com/case-studies/turner-construction-uses-ground-robots-to-cut-inspection-time-by-over-95-percent/>, accessed 28-09-25
- [8] ‘RATP: Inspections after dark | Boston Dynamics’, <https://bostondynamics.com/case-studies/ratp/>, accessed 28-09-25
- [9] ‘Virginia Tech | Boston Dynamics’, <https://bostondynamics.com/case-studies/virginia-tech/>, accessed 28-09-25
- [10] ‘Acciona | Boston Dynamics’, <https://bostondynamics.com/case-studies/acciona/>, accessed 28-09-25
- [11] ‘Pomerleau | Boston Dynamics’, <https://bostondynamics.com/case-studies/pomerleau/>, accessed 28-09-25
- [12] ‘Brasfield & Gorrie | Boston Dynamics’, <https://bostondynamics.com/case-studies/brasfield-gorrie/>, accessed 28-09-25
- [13] ‘Foster + Partners | Boston Dynamics’, <https://bostondynamics.com/case-studies/foster-partners/>, accessed 28-09-25
- [14] ‘Swinerton | Boston Dynamics’, <https://bostondynamics.com/case-studies/swinerton/>, accessed 28-09-25
- [15] ‘Spot SDK — Spot 5.0.1 documentation’, <https://dev.bostondynamics.com/readme#>, accessed 29-09-25
- [16] ‘gRPC’, <https://grpc.io/>, accessed 29-09-25
- [17] ‘Spot’, <https://support.bostondynamics.com/s/topic/0TOUS00000003syL4AQ/spot>, accessed 29-09-25
- [18] ‘Spot Core I/O | Boston Dynamics’, <https://bostondynamics.com/products/payload/spot-core-io/>, accessed 29-09-25
- [19] ‘RICOH THETA Z1’, <https://thetaz1.com/en/>, accessed 29-09-25
- [20] ‘abhishekdilipatil/spot-sdk: Spot SDK repo’, <https://github.com/abhishekdilipatil/spot-sdk>, accessed 29-09-25

Appendix

Appendix A : gRPC

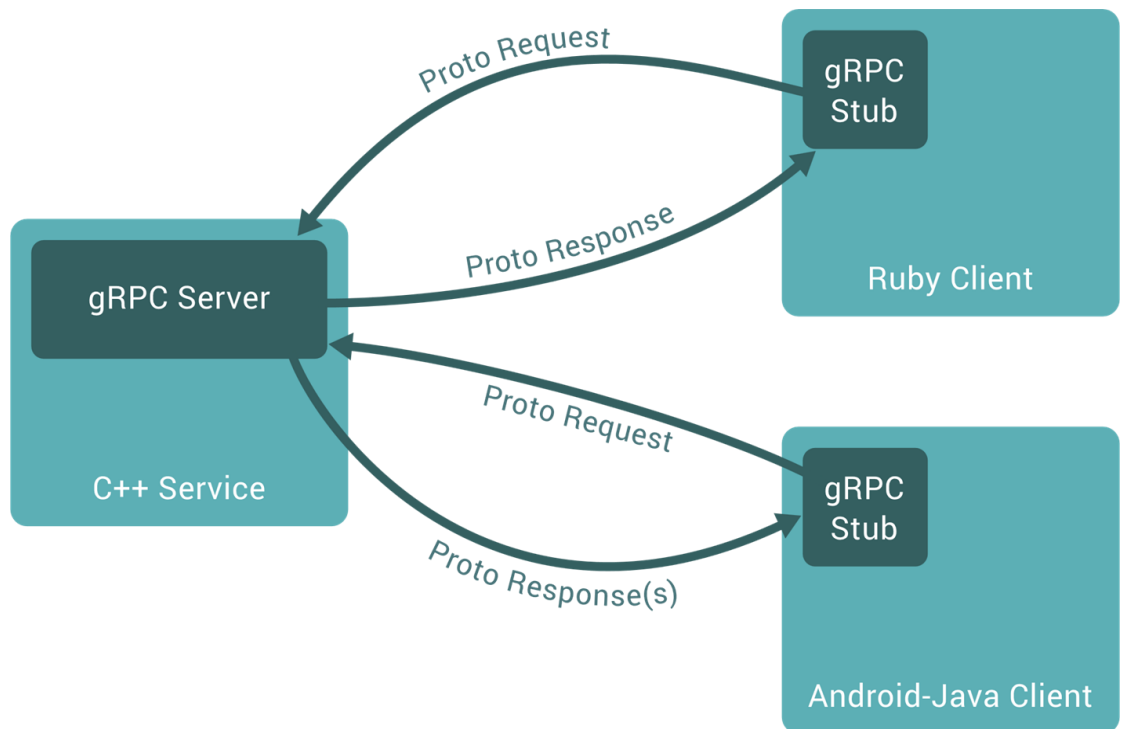


Figure 15: Communication using gRPC (block diagram)

gRPC has four kinds of service methods:

- Unary RPCs where the client sends a single request to the server and gets a single response back, just like a normal function call.

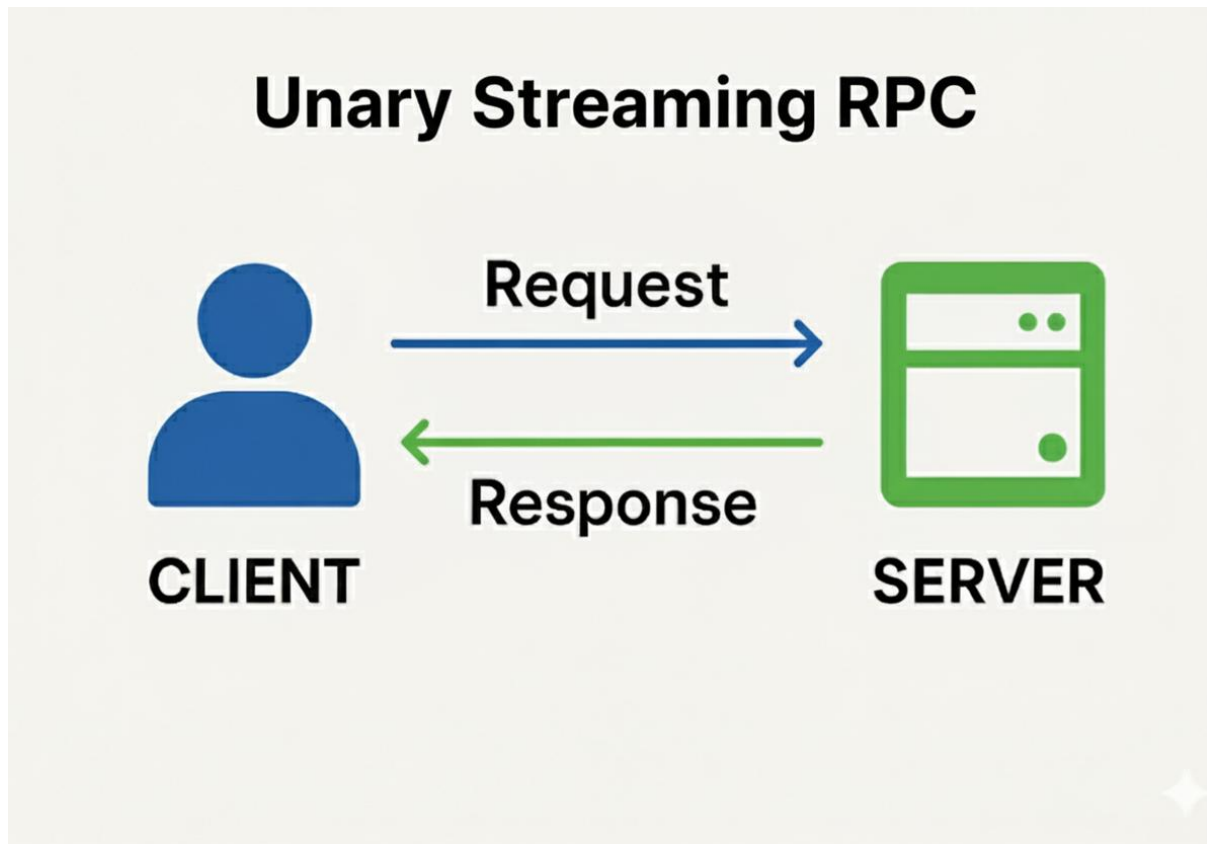


Figure 16: Unary Streaming RPC, image source: ChatGPT

- Server streaming RPCs where the client sends a request to the server and gets a stream to read a sequence of messages back. The client reads from the returned stream until there are no more messages. gRPC guarantees message ordering within an individual RPC call.

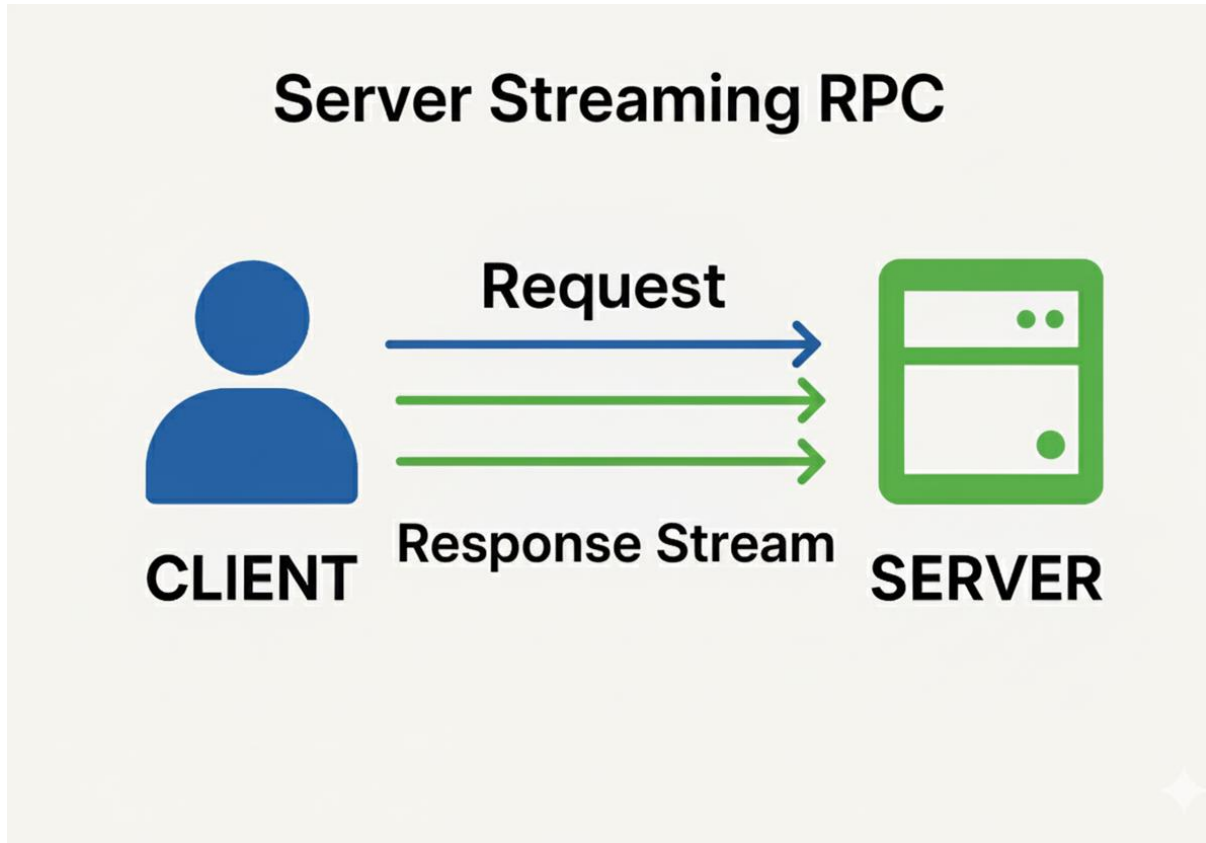


Figure 17: Server Streaming RPC, image source: ChatGPT

- Client streaming RPCs where the client writes a sequence of messages and sends them to the server, again using a provided stream. Once the client has finished writing the messages, it waits for the server to read them and return its response. Again, gRPC guarantees message ordering within an individual RPC call.

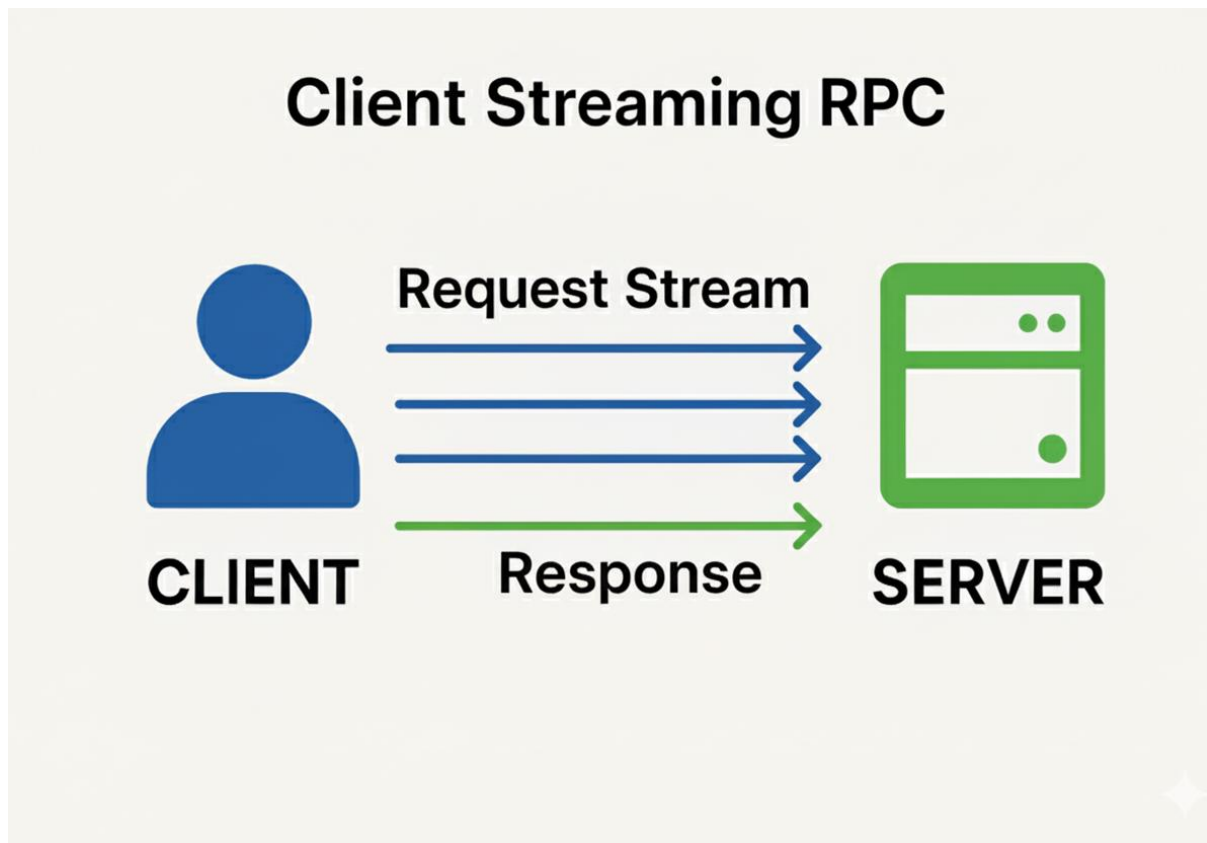


Figure 18: Client Streaming RPC, image source: ChatGPT

- Bidirectional streaming RPCs where both sides send a sequence of messages using a read-write stream. The two streams operate independently, so clients and servers can read and write in whatever order they like: for example, the server could wait to receive all the client messages before writing its responses, or it could alternately read a message then write a message, or some other combination of reads and writes. The order of messages in each stream is preserved.

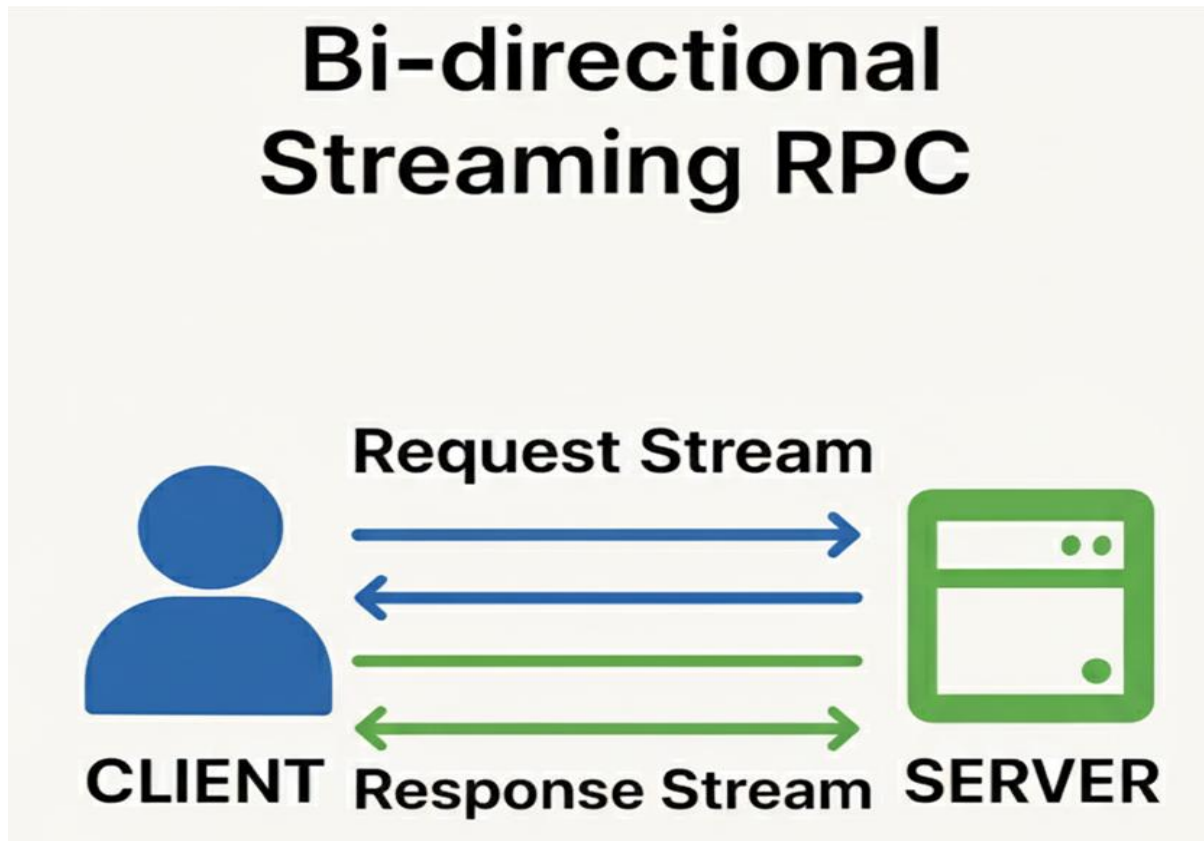


Figure 19: Bi - directional RPC, image source: ChatGPT

gRPC specifies the service interfaces and remote procedure calls (RPCs) supported by the service. For example, the Spot API has an authentication service to check the passed in username and password and returns a session token if they are valid.

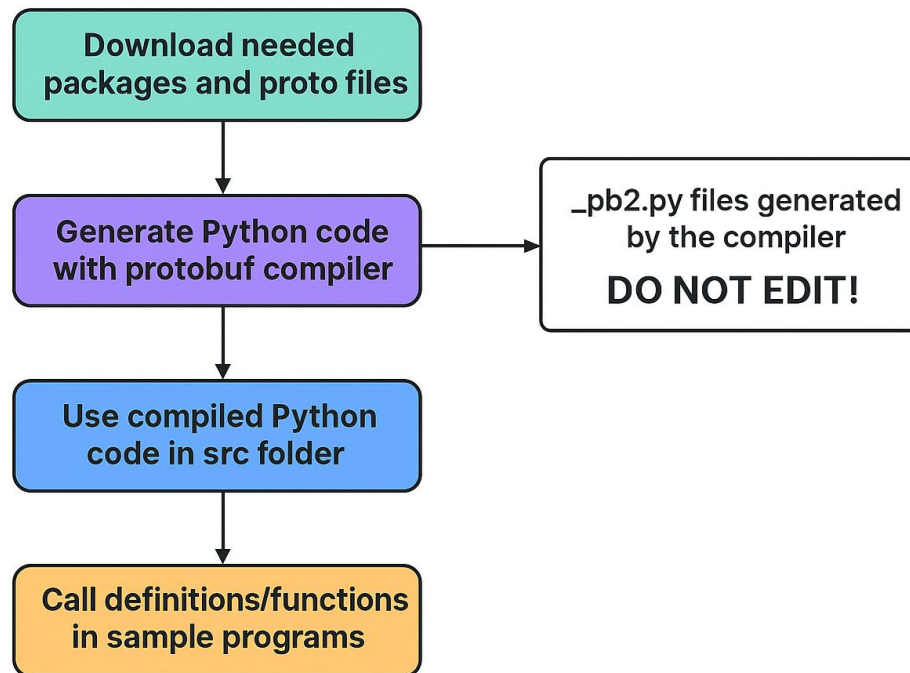


Figure 20: Working with RPC and SDK imports, image source: ChatGPT

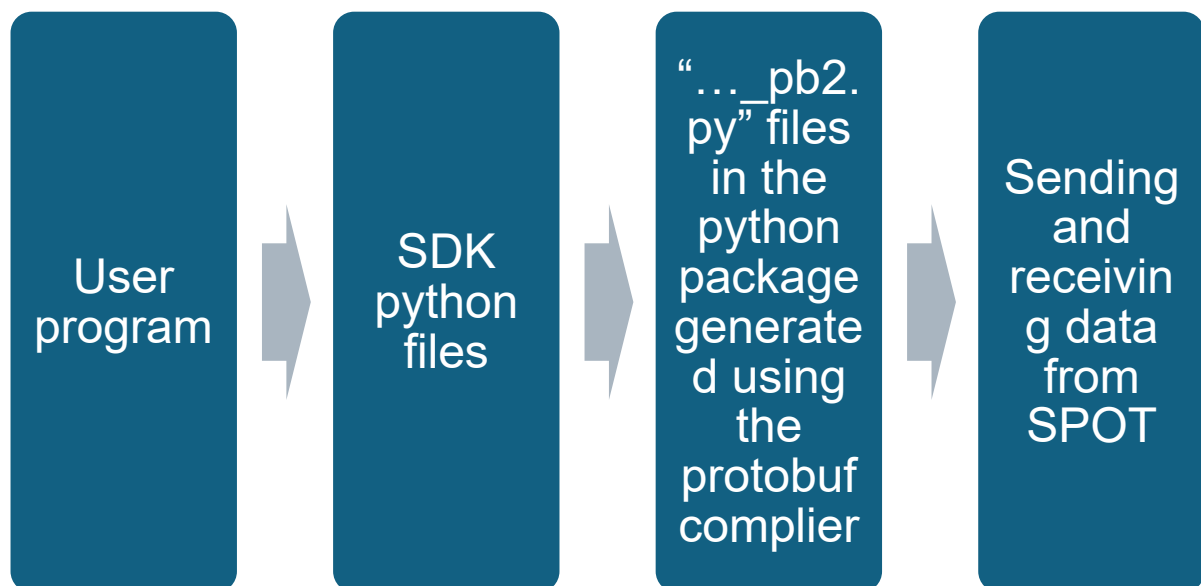


Figure 21: Flow chart of the process at the client end

Appendix B : SPOT hardware specifications

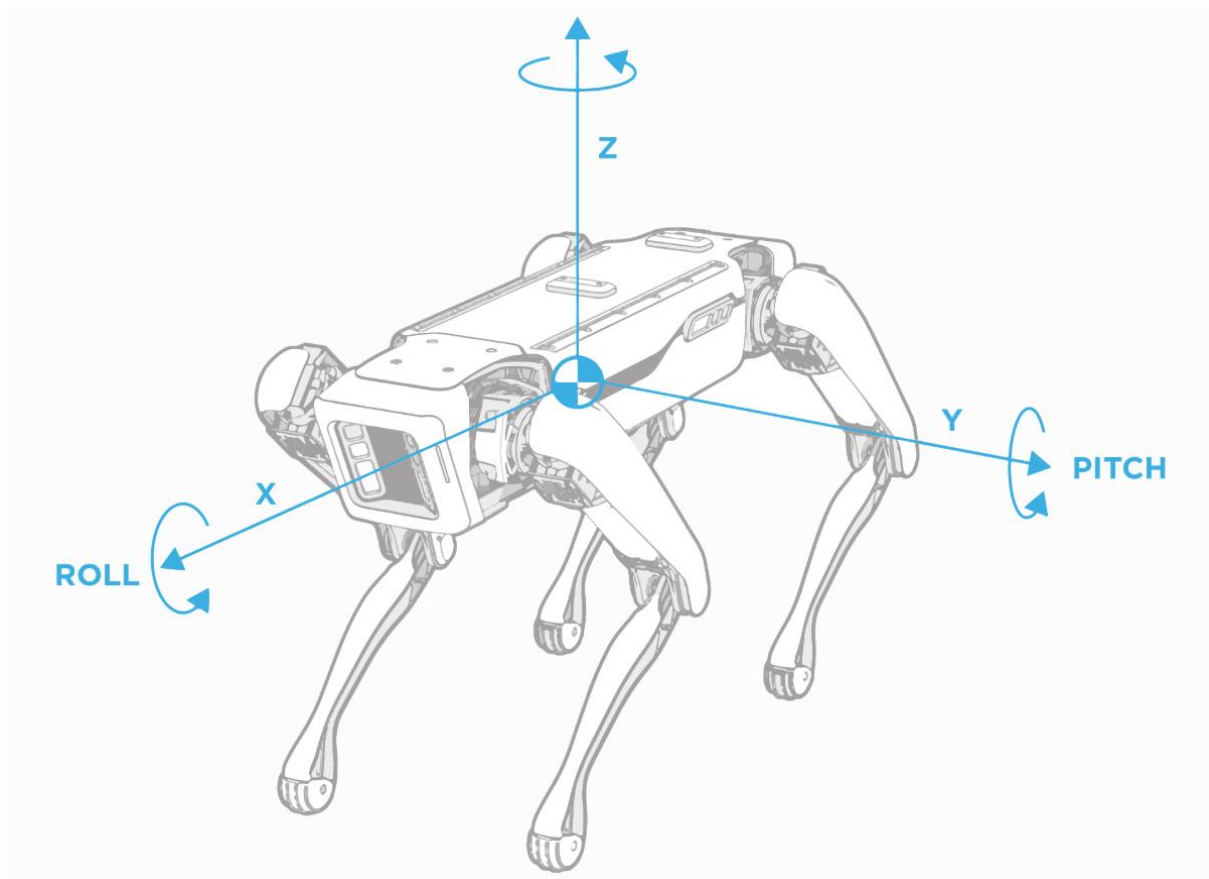


Figure 22: SPOT axes that are used for motion definition

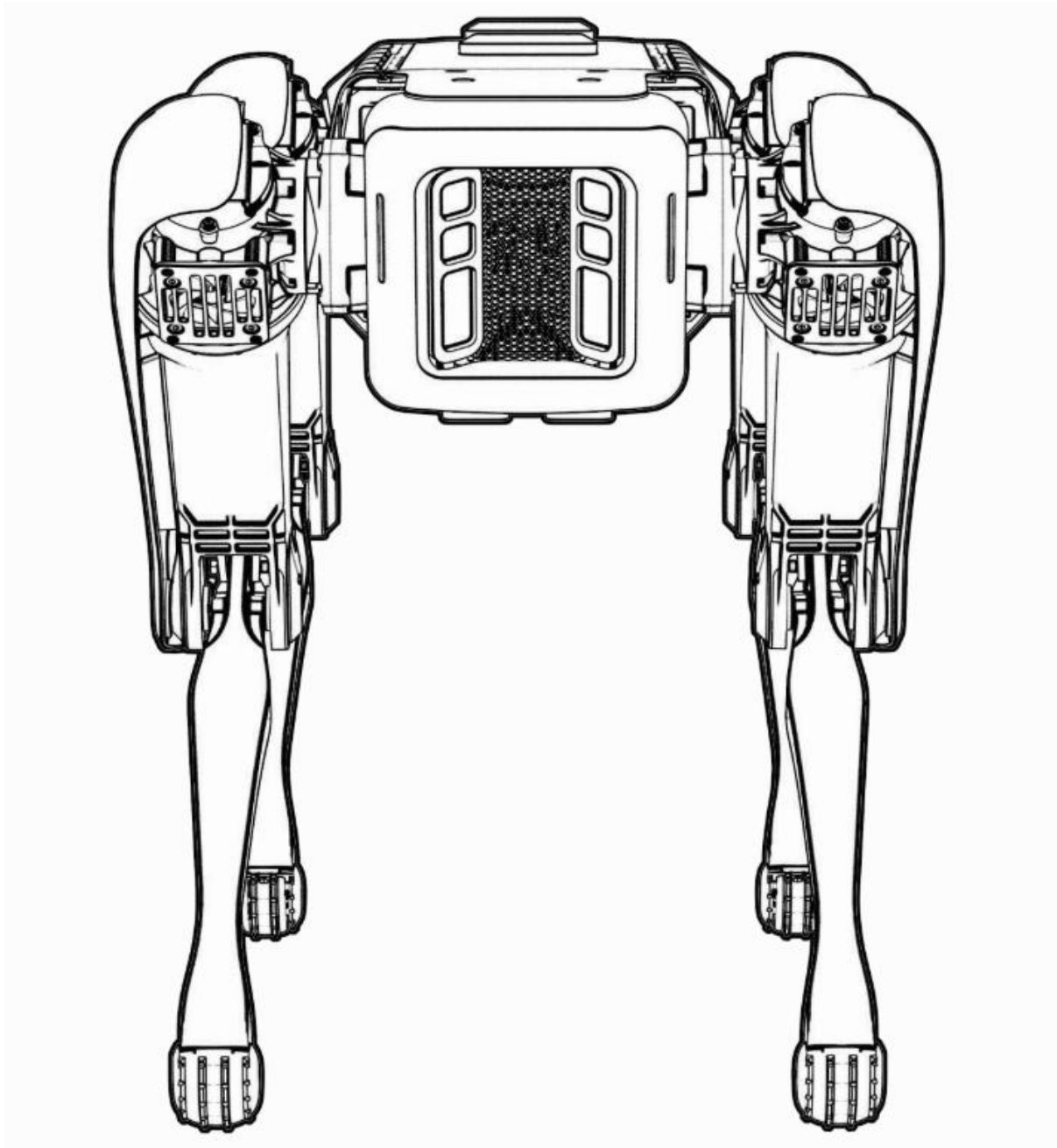


Figure 23: Front view of SPOT

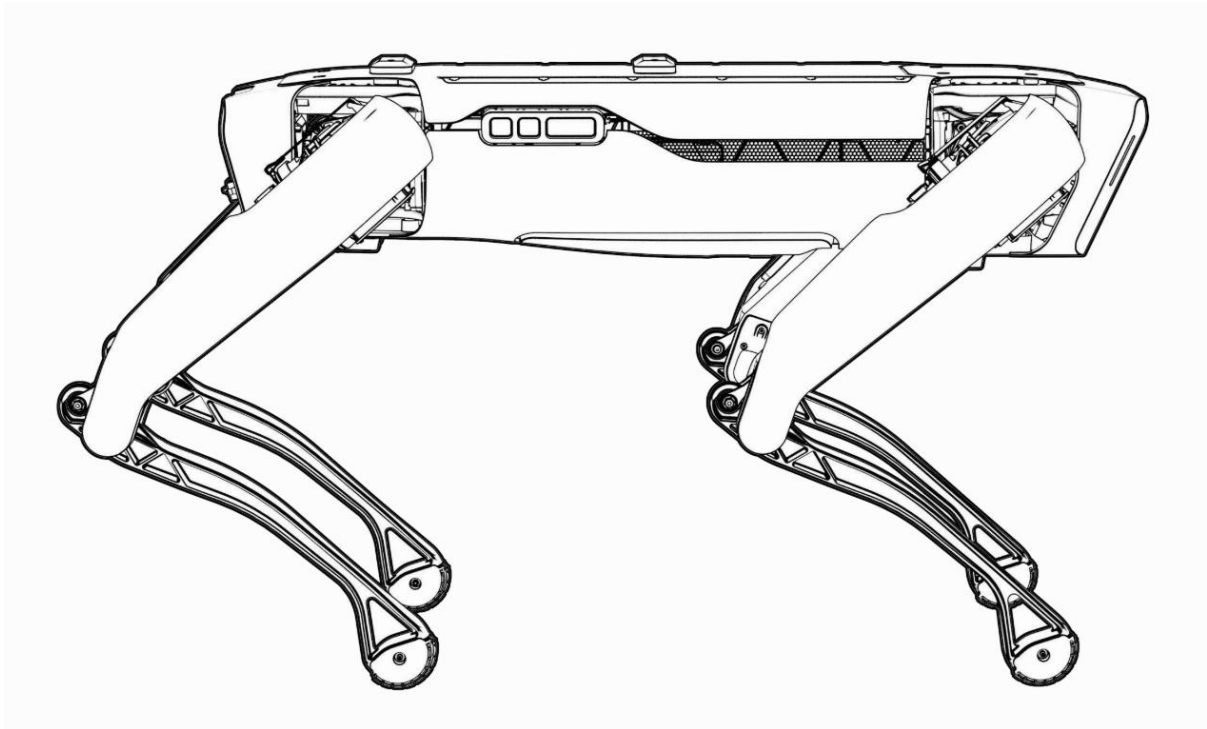


Figure 24: Side view of SPOT

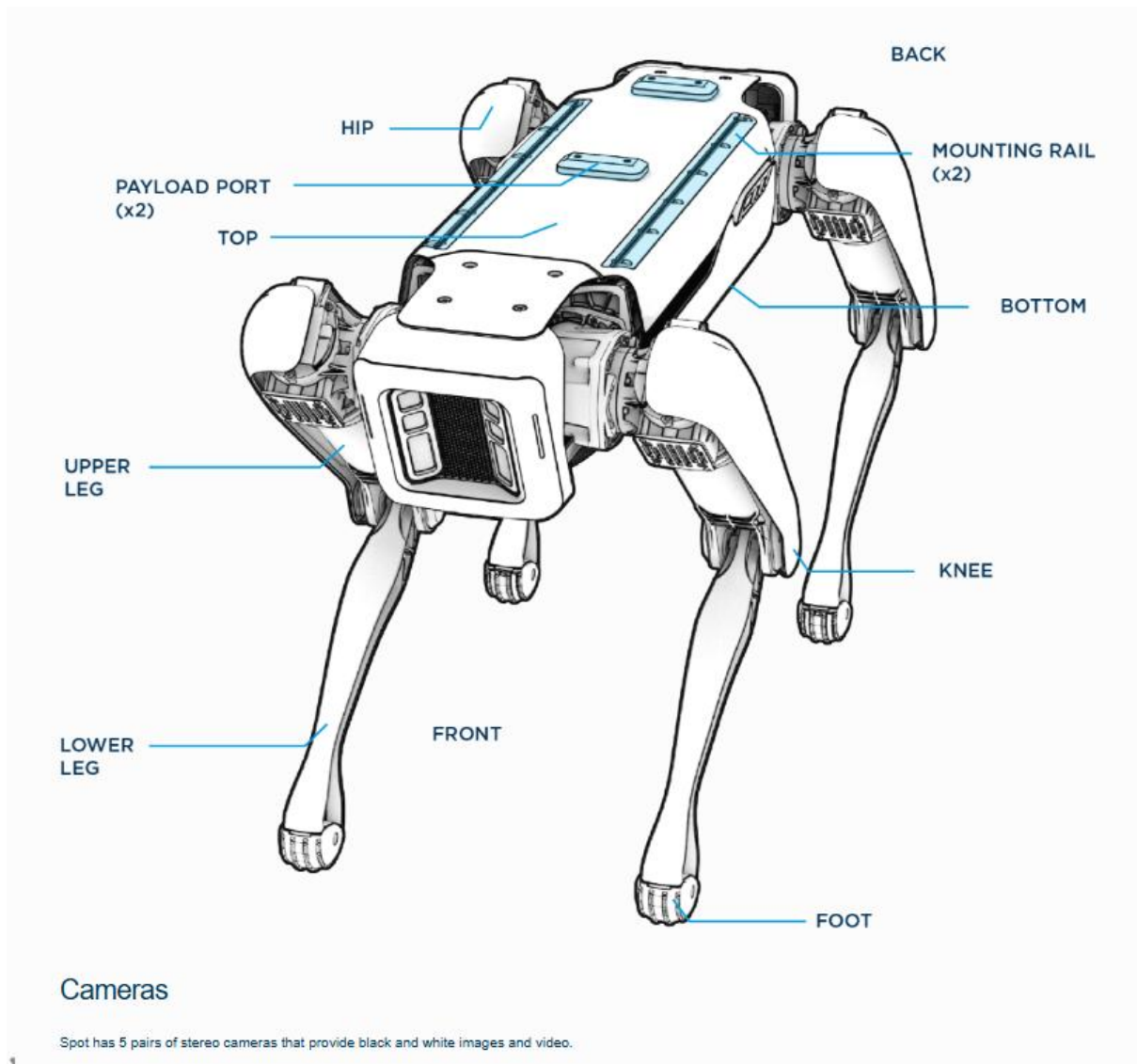


Figure 25: Basic labelled diagram of SPOT

Specification	Value
Length	1100 mm
Width	500 mm
Height (standing)	840 mm
Height (Seating)	191 mm
Weight	32.5 kg
DOF	12
Speed	1.6 m/s
Ingress protection	IP54
Operating Temperature	-20 C to 45 C
Slopes	+/- 30 Degrees
Max step height	300 mm
Lighting	Above 2 lux
Battery capacity	605 Wh
Max battery voltage	58.8V
Typical runtime	90 minutes
Standby runtime	180 minutes
Charger power	400W
Max charge current	7A
Time to charge	120 minutes
Payload max weight	14 kg
Max power per port	150 W
Payload ports	2
Camera type	Projected stereo
Field of view	360 Degrees
Operating range	4 meters
Wi-Fi	802.11
Ethernet	1000Base-T

Table B-1: Hardware specification of SPOT, necessary to understand the working environment of SPOT.

1. Initiation Checklist

- a. Clear the area & brief people nearby.
- b. Ensure an operator has a reachable hardware or software E-Stop.
- c. Power and battery check.
- d. Put the tablet or your control laptop on the same network as the robot.

2. Programming Operations

- a. Install the Python SDK
 - i. Install Python 3.7–3.10.
 - ii. Verify your python install is the correct version. Open a command prompt or start your python IDE: Then enter
\$ python --version
Python 3.10.12
 - iii. Windows users: There are two common methods for starting the Python interpreter on the command line. First, launch a terminal: Start > Command Prompt. At the command prompt, enter either:
> python.exe Or > py.exe

- b. Pip Installation Pip is the package installer for Python. The Spot SDK and the third-party packages used by many of its programming examples use pip to install.

Check if pip is installed by requesting its version:

```
$ python3 -m pip --version
pip 19.2.1 from <PATH_ON_YOUR_COMPUTER>
```

For Windows users:

```
> py.exe -3 -m pip --version 3.
```

- c. Install Spot Python packages

With python and pip properly installed and configured, the Python packages are easily installed or upgraded from PyPI with the following command:

```
$ python3 -m pip install --upgrade bosdyn-client bosdyn-mission bosdyn-choreography-client bosdyn-orbit
```

Installing the bosdyn-client, bosdyn-choreography-client and bosdyn-mission packages will also install bosdyn-api and bosdyn-core packages with the same version. The command above installs the latest version of the packages.

- d. Verify your Spot packages installation Enter this command to verify all the packages

```
$ python3 -m pip list --format=columns | grep bosdyn
```

The Output Should be:

bosdyn-api	5.0.1
bosdyn-choreography-client	5.0.1
bosdyn-choreography-protos	5.0.1
bosdyn-client	5.0.1
bosdyn-core	5.0.1
bosdyn-mission	5.0.1
bosdyn-orbit	5.0.1

For Windows Users:

```
> python3 -m pip list --format=columns | findstr bosdyn
```

For more accurate step and troubleshooting you can refer to:
<https://dev.bostondynamics.com/docs/python/quickstart>